

Beat Tracking as Latent Order-Preserving Alignment

Moon-Ryul Jung
Dept of Art & Technology
Sogang University

August 28, 2026

Abstract

We formulate beat and downbeat detection as a problem of latent order-preserving alignment: a fixed number of candidate predictors independently regress candidate event times and classify them, deliberately overgenerating more candidates than there are true events, and training must decide which candidates align to a genuine event and which align to nothing — the same role a blank symbol plays in Connectionist Temporal Classification’s own alignment. This is a latent-variable alignment problem in the same family as CTC and Dynamic Time Warping, both of which replace a search over an exponential space of possible alignments with a monotonicity-constrained dynamic program; the correspondence developed here shares their precise structure, a deterministic lattice of legal moves scored entirely by local emission cost, carrying no independent transition weight of its own. We show that once the ground-truth event sequence is indexed by its natural temporal order, and the candidate times are made strictly increasing by an architectural reparameterization, this latent alignment problem reduces to a choice of *which subset of candidates is active* — an order-constrained subset-selection problem, since the pairing between the active candidates and the sorted ground truth is then forced rather than searched over. This problem is solved exactly by an $O(NM)$ dynamic program. We state the conditions under which the resulting solution is optimal without qualification, and the conditions under which the order constraint is a deliberate modeling choice rather than a free simplification, and we specify the resulting training loss. We further diagnose why downbeat detection is systematically harder than beat detection under this and other formulations, and propose three targeted adjustments — a candidate-level self-attention pass restricted to the classification branch, class-specific loss reweighting, and a differentiable bar-length periodicity regularizer — to mitigate the gap. This formulation is presented strictly within the discriminative paradigm; it is architecturally and probabilistically distinct from the generative bar-pointer Dynamic Bayesian Network framework and the two should not be conflated in method sections addressing either model.

1 Introduction

Classical beat and downbeat trackers, including Dynamic Bayesian Network (DBN) decoders operating over a bar-pointer state space (Böck, Krebs & Widmer, 2016), treat beat tracking as a left-to-right sequential inference problem: a hidden tempo-and-phase state evolves according to a Markov transition law, and the beat sequence is recovered by Viterbi decoding through this state space. This generative formulation enforces rhythmic regularity by construction, since the transition law directly penalizes implausible tempo changes.

An alternative, purely discriminative formulation asks an encoder to produce a fixed number N of candidate events directly, one per position in a coarse partition of the input fragment, each independently emitting a candidate time and a class label. N is set from a physical bound on tempo

rather than from the number of true events in any given fragment, so the architecture deliberately overgenerates: most candidates are expected to represent no event at all.

Overgeneration creates a correspondence problem. The N candidates carry no built-in notion of which one, if any, should represent which true event; nothing in the architecture privileges one candidate over another for a given ground-truth event ahead of training. Before a differentiable loss can be computed, something must decide which candidates are held responsible for the true events, which are held responsible for producing nothing, and what target each responsible candidate is compared against. This decision is itself the value of an unobserved quantity — a correspondence σ , formalized in Section 4, that is never given as data and must instead be inferred: σ is a genuine latent function, not a hyperparameter or an architectural output, and treating it as such is what licenses the training procedure of Section 7. This paper works out that correspondence procedure directly from the structure of the beat-tracking problem: ground-truth beat and downbeat events are strictly ordered in time, and we show that imposing the corresponding order on the candidates reduces the correspondence problem to selecting an active subset of candidates, solvable exactly and efficiently.

This latent-alignment view is not specific to beat tracking. A structurally identical problem — deciding, given a set of raw candidate positions, which ones align to a real event and which align to nothing, as a latent variable inferred rather than observed — is the same problem CTC and DTW solve for sequence alignment more generally: both replace a search over an exponential space of possible alignments with a monotonicity-constrained dynamic program, and CTC in particular already has a native mechanism, its blank symbol, for a position that aligns to nothing at all. Section 11 develops this connection precisely, including the specific sense in which this construction shares CTC’s and DTW’s own emission-only structure, not merely their asymptotic complexity.

2 Problem Setup and Notation

Let $x = (x_1, \dots, x_T)$ be a log-mel spectrogram computed from a fragment of fixed duration D , with $x_t \in \mathbb{R}^{n_{\text{mel}}}$ the spectrogram frame at hop t and $T = D/\text{hop}$ the resulting number of frames. The ground-truth annotation is M events

$$y_1, \dots, y_M, \quad y_i = (c_i, t_i), \quad c_i \in \{0, 1, 2\} = \{\text{DB}, \text{B}, \text{B}_*\}, \quad t_i \in [0, 1],$$

indexed in ascending temporal order, $t_1 < \dots < t_M$. Here $0 = \text{DB}$ (downbeat), $1 = \text{B}$ (beat, confirmed *not* a downbeat), and $2 = \text{B}_*$ denotes a beat-only annotation — some beat occurred, downbeat status unchecked — a distinct label from B , not a reuse of it: B asserts non-downbeat status, B_* asserts nothing beyond “some beat.” A fully-labeled track records $c_i \in \{\text{DB}, \text{B}\}$ only, choosing correctly between them; a beat-only-labeled track records $c_i = \text{B}_*$ for every one of its M events. A fixed integer $N > M$ denotes the number of candidates the network always produces, deliberately overgenerating relative to M ; its exact value is derived, not chosen directly, in Section 3 once the downsampling schedule is fixed, from a physical tempo lower bound $N_{\min} := \text{BPM}_{\max} \times D_{\min}$ (we use $\text{BPM}_{\max} = 200$, $D = 30\text{s}$, giving $N_{\min} = 100$).

Because the model always emits exactly N candidates while the fragment contains $M < N$ true events, training requires deciding which M of the N candidates are held responsible for the true events. We show that once the ground-truth sequence is sorted (given) and the candidate times are made strictly increasing by construction (Section 3), this correspondence reduces to a choice of *which subset of candidates is active* — an order-preserving injection $\sigma : \{1, \dots, M\} \rightarrow \{0, \dots, N-1\}$, $\sigma(1) < \dots < \sigma(M)$. Write $\mathfrak{D}_{M,N}$ for the set of such functions; $|\mathfrak{D}_{M,N}| = \binom{N}{M}$, since σ is fully determined by its image.

σ is treated as a genuinely latent variable throughout — never observed, not part of θ , and requiring inference from θ, x, y . It is worth flagging one indexing choice explicitly here, since it recurs in Section 8: σ is a function *of* the M ground-truth event indices $i = 1, \dots, M$ (it is $\sigma(i)$ that is defined, not something indexed directly by candidate j), so σ ’s own domain is event-indexed, not candidate-indexed. This matters because it means σ has no value at all for a candidate outside its image — there is no meaningful sense in which an unmatched candidate $j \notin \text{im}(\sigma)$ “has” a σ -coordinate, since σ was never defined over candidates to begin with.

What follows makes precise how σ enters the model’s likelihood — and, just as importantly, how it does not.

Fragments longer than D . A track’s own duration is generally much longer than D , so a full piece must be tiled into multiple overlapping fragments of duration D , each processed independently by everything in Sections 3–8. We fix the convention here since Section 9.3 depends on it: consecutive fragments are tiled with a fixed overlap of 2β on each side, for a border duration $\beta \in (0, D/2)$ (we use $\beta = 120\text{ms}$ as a starting point, unvalidated), and only the central $[\beta, D - \beta]$ portion of each fragment’s own decoded output is kept once decoding is complete — the overlapping edges exist so that a candidate near a fragment’s own boundary is never the *only* candidate covering that moment in the piece, not so that its output is itself trusted. σ itself is never defined across fragments: it is a correspondence internal to one fragment’s own M, N and candidate indexing, with no meaning for a candidate belonging to a different fragment. Section 9.3 returns to this once decoding, rather than the correspondence problem itself, is the topic.

3 Prediction Architecture and Monotonic Reparameterization

We assume, as is standard in current beat-tracking architectures, that the encoder is a transformer: a stack of self-attention layers mapping x to a feature sequence $h = (h_1, \dots, h_T)$ in which every h_t has, in principle, access to the entire input fragment through self-attention, rather than only to a local receptive field as in a convolutional or recurrent encoder. This assumption matters for the downbeat-specific discussion of Section 10: the frame-level features h_t are already globally contextualized, so any residual difficulty in recovering bar-level (as opposed to beat-level) structure downstream is not attributable to a limited receptive field, and must be addressed elsewhere in the architecture.

The encoder output has length T , generally different from N , so some fixed operator is needed to produce the N candidate feature vectors $z_0, \dots, z_{N-1}$ from h . We write this simply as

$$z_j(\theta, x) = \text{Downsample}(h)_j, \quad j = 0, \dots, N - 1,$$

for any fixed downsampling operator reducing a length- T sequence to a length- N sequence, for instance a strided convolution or a strided average-pooling layer. Nothing in Sections 4 through 10 depends on which such operator is used; it is an implementation choice orthogonal to the correspondence problem, the dynamic program, and the loss, and we do not assign it any geometric or anchoring role. In particular, no candidate is associated with a fixed reference time, and equation (1) below does not reference the position that produced z_j : z_j is a genuine function of the input x , but it is not required to localize $\hat{t}_j$. Each z_j is passed through two prediction heads: a classification head returning logits over $\{\text{DB}, \text{B}, \emptyset\}$, and a scalar regression head returning a raw value $r_j(\theta, x) \in \mathbb{R}$.

We compute the predicted time sequence by a global monotonic reparameterization of the raw regression outputs $r_0, \dots, r_{N-1}$:

$$\hat{t}_j(\theta, x) = \frac{\sum_{k=0}^j \text{softplus}(r_k(\theta, x))}{\sum_{k=0}^{N-1} \text{softplus}(r_k(\theta, x))}, \quad j = 0, \dots, N-1, \quad (1)$$

where $\text{softplus}(r) = \log(1 + e^r) > 0$ for every $r \in \mathbb{R}$. Because each summand in the numerator is strictly positive, the sequence is strictly increasing for every value of θ and every input x :

$$\hat{t}_0(\theta, x) < \hat{t}_1(\theta, x) < \dots < \hat{t}_{N-1}(\theta, x), \quad \text{for all } \theta, x,$$

and the normalization by the full sum places every $\hat{t}_j$ in $(0, 1)$. Monotonicity is therefore an architectural invariant rather than a property that must be learned or penalized into the network: the network cannot represent a crossing hypothesis, regardless of what values $r_0, \dots, r_{N-1}$ take. Write $\hat{p}_j(\cdot \mid x; \theta)$ for the predicted class distribution of candidate j over $\{\text{DB}, \text{B}, \emptyset\}$.

Where $\hat{t}_j$ ends up is determined entirely by training, not by any property of z_j 's extraction. For a matched candidate ($j = \hat{\sigma}(i)$ for some i), the $L1$ term in the training loss (8) directly pulls $\hat{t}_j$ toward the ground-truth time t_i . For an unmatched candidate ($j \notin \text{im}(\hat{\sigma})$), the loss supervises only its class output toward $\emptyset$ and places no target on $\hat{t}_j$ at all, so its predicted time is unconstrained beyond the ordering in equation (1). Whichever downsampling operator produces z_j therefore supplies, at most, an inductive bias on the raw output r_j through whatever locality that operator has — not a constraint on the output, and not a claim that candidate j corresponds to any particular time. Whether a local downsampling operator helps or hurts relative to one with a more global receptive field is an empirical question that the architecture leaves open, not one it settles by construction.

4 The Correspondence Problem

Because the model always emits exactly N candidates and the ground-truth fragment contains $M < N$ true events, training requires deciding which M of the N candidates are held responsible for the true events, and which class and time target each is compared against. Rather than combine a classification penalty and a time penalty by an unmotivated weighted sum, we derive the per-pair cost as the negative log-likelihood of a stated observation model, so that every term in it has an explicit probabilistic meaning.

Fix a candidate j and a ground-truth event $y_i = (c_i, t_i)$, and suppose y_i were generated by two conditionally independent channels reading off candidate j 's outputs: a categorical channel for the class, and a channel treating the true time t_i as a virtual observation of candidate j 's predicted time $\hat{t}_j(\theta, x)$, corrupted by noise of a stated distribution rather than compared to it by an arbitrary norm.

$$C_i \mid \hat{p}_j \sim \text{Categorical}(\hat{p}_j(\cdot \mid x; \theta)), \quad T_i \mid \hat{t}_j \sim \text{Laplace}(\hat{t}_j(\theta, x), b), \quad C_i \perp T_i \mid \hat{p}_j, \hat{t}_j, \quad (2)$$

with Laplace density $f(t_i \mid \hat{t}_j) = \frac{1}{2b} \exp(-|t_i - \hat{t}_j(\theta, x)|/b)$ and fixed scale $b > 0$. The conditional independence in (2) is stated as an assumption, not derived from anything prior, and it is what licenses factorizing the joint density of y_i into a product of the two channels below. The choice of Laplace rather than Gaussian for the time channel is not a free modeling choice made to fit a predetermined answer: Laplace is the unique distribution whose negative log-density is, up to an additive constant, the $L1$ penalty already required of this term, so it is the distribution forced by the requirement that the resulting negative log-likelihood reproduce an $L1$, rather than an $L2$, regression cost.

Under (2), the joint density of observing y_i given a match to candidate j is

$$p(y_i | \hat{y}_j) = \hat{p}_j(c_i | x; \theta) \cdot \frac{1}{2b} \exp\left(-\frac{|t_i - \hat{t}_j(\theta, x)|}{b}\right),$$

and its negative log is

$$-\log p(y_i | \hat{y}_j) = -\log \hat{p}_j(c_i | x; \theta) + \frac{1}{b} |t_i - \hat{t}_j(\theta, x)| + \log(2b).$$

The additive constant $\log(2b)$ does not depend on θ while b is held fixed, so it may be dropped without affecting the location of any minimizer over θ . Identifying $\lambda_{L1} := 1/b$, we take the per-pair cost to be exactly this negative log-likelihood:

$$\mathcal{L}_{\text{match}}(y_i, \hat{y}_j) = -\log \hat{p}_j(c_i | x; \theta) + \lambda_{L1} |t_i - \hat{t}_j(\theta, x)|, \quad (3)$$

which is now, term for term, $-\log p(y_i | \hat{y}_j)$ under (2), rather than an engineered sum. In particular λ_{L1} is not a free balancing weight chosen for convenience; it is the precision (inverse scale) of the assumed time-observation noise, and could in principle be estimated from data — e.g. by a closed-form update setting b to the mean absolute matched residual, in the same spirit as the learnable transition-bandwidth parameters in a bar-pointer DBN — rather than fixed as a hyperparameter, though we do not pursue a learnable b here. This derivation also fixes a discrepancy in the raw-probability convention inherited from the object-detection matching-cost literature: only $-\log \hat{p}_j(c_i | x; \theta)$, not $-\hat{p}_j(c_i | x; \theta)$, is licensed by the observation model in (2), so we use $-\log \hat{p}_j$ throughout, making the per-pair cost and the training loss of Section 7 the same quantity in form, not merely in spirit.

The ground-truth events are sorted by construction (Section 2), and the candidate times are sorted by construction (Section 3, equation (1)). Given two sorted sequences, the only remaining freedom in a correspondence between them is which elements of the longer sequence participate: once a set of M candidates is chosen, pairing the i -th ground-truth event with the i -th smallest chosen candidate is the only pairing consistent with both orderings. The correspondence problem is therefore not a search over pairings at all; it is a search over which candidates to select — and that search is a search over the value of a single unobserved object, σ , defined next.

Definition 1. *An order-preserving injection is a strictly increasing function $\sigma : \{1, \dots, M\} \rightarrow \{0, \dots, N-1\}$, i.e. $\sigma(1) < \sigma(2) < \dots < \sigma(M)$. Write $\mathfrak{D}_{M,N}$ for the set of all such functions. Any $\sigma \in \mathfrak{D}_{M,N}$ pairs the i -th ground-truth event with candidate $\sigma(i)$: $y_i \leftrightarrow \hat{y}_{\sigma(i)}$, for $i = 1, \dots, M$.*

σ is a latent function in the ordinary statistical sense of that term: it is never observed, not part of θ , not an architectural output, and not given by the annotation, which supplies only $y_1, \dots, y_M$ themselves, not which candidate stands for which. Everything from this point through Section 7 is the construction of a well-defined estimation problem for this one unobserved function, given the observed $y_{1:M}$ and the candidate predictions $\hat{y}_0, \dots, \hat{y}_{N-1}$ that θ and x already determine.

Although σ is a function, $\mathfrak{D}_{M,N}$ is not the space of all injections or permutations: because σ must be strictly increasing, it is completely determined by its image $\{\sigma(1), \dots, \sigma(M)\} \subset \{1, \dots, N\}$, so $|\mathfrak{D}_{M,N}| = \binom{N}{M}$ — the same search space size as choosing which M candidates participate, not $M!$ or any larger permutation-sized count. The estimation problem is

$$\hat{\sigma} = \arg \min_{\sigma \in \mathfrak{D}_{M,N}} \sum_{i=1}^M \mathcal{L}_{\text{match}}(y_i, \hat{y}_{\sigma(i)}). \quad (4)$$

4.1 Learning the Precision Parameter

Fixing b (equivalently $\lambda_{L1} = 1/b$) as a hyperparameter discards information the training data actually carries: b is the scale of the assumed time-observation noise in (2), and its maximum-likelihood value is computable in closed form from the residuals the model itself produces. Recall from Section 2 that x , M , and N are all specific to one fragment of fixed duration D ; everything below is computed for that single fragment, not pooled across the dataset except where a minibatch average is stated explicitly. Given the matched pairs $(t_i, \hat{t}_{\hat{\sigma}(i)}(\theta, x))$ for $i = 1, \dots, M$, with $\hat{\sigma}$ held fixed as in Section 7, the time-channel contribution to the joint negative log-likelihood of this fragment is

$$M \log(2b) + \frac{1}{b} \sum_{i=1}^M |t_i - \hat{t}_{\hat{\sigma}(i)}(\theta, x)|.$$

Differentiating with respect to b and setting the result to zero gives

$$\hat{b} = \frac{1}{M} \sum_{i=1}^M |t_i - \hat{t}_{\hat{\sigma}(i)}(\theta, x)|, \quad (5)$$

the sample mean absolute residual over matched pairs — the classical maximum-likelihood scale estimate for a Laplace distribution. In practice (5) is computed once per minibatch, or as an exponential moving average across minibatches for stability, and held fixed while θ is updated by gradient descent using the resulting b in (3) and (8). This is an ordinary closed-form maximum-likelihood point estimate of a single scalar parameter, not an expectation over a latent variable’s posterior, and there is no distribution over b being marginalized; it should not be described by EM terminology. What genuinely carries over from Section 7’s treatment of $\hat{\sigma}$ is only the alternation pattern — one block of quantities held fixed while another is optimized to an exact optimum, then swapped — which is more accurately described as block coordinate ascent on (θ, b) .

4.1.1 Why a shared, global b does not admit cheating

Under (5), b is a single scalar shared by every candidate in every fragment of the minibatch, and it is updated only from the aggregate residual, not from any individual pair. No single candidate’s prediction can move $\hat{b}$ enough to materially reduce its own penalty: within any block of gradient steps on θ , b is held fixed, so a poorly localized candidate still incurs $|t_i - \hat{t}_{\hat{\sigma}(i)}|/b$ at whatever b currently is, and the only way to reduce that term is to reduce the residual itself. There is no route by which the network can unilaterally inflate or shrink b to excuse an individual bad prediction; b moves only in response to a genuine, aggregate change in localization accuracy across many matched pairs.

4.1.2 Where cheating becomes possible: per-candidate precision

The situation changes if precision is instead predicted per candidate, $b_j = b_j(\theta, x)$, from a small head reading z_j — a natural extension if one wants the model to express reduced confidence in acoustically ambiguous passages rather than a single dataset-wide noise level. This reopens exactly the degree of freedom the global version closes, and it does so in two directions, both traceable to the same term in the loss.

Holding a single candidate’s residual $|t_i - \hat{t}_j|$ fixed, the per-example time-channel loss is $\ell(b_j) = |t_i - \hat{t}_j|/b_j + \log(2b_j)$, with $\partial\ell/\partial b_j = -|t_i - \hat{t}_j|/b_j^2 + 1/b_j$. Two failure modes follow directly:

Precision inflation on hard candidates. Whenever the current residual exceeds the current precision scale, $|t_i - \hat{t}_j| > b_j$, the gradient $\partial\ell/\partial b_j$ is negative: increasing b_j reduces the loss. If this

is an easier direction for gradient descent to move in than reducing the residual itself — plausible early in training, or for a genuinely hard example — the network is rewarded for declaring low confidence rather than for improving $\hat{t}_j$. This is the cheating the precision parameter is exposed to: the model pays a much smaller, easily-purchased penalty by inflating b_j instead of the harder work of localizing correctly. It is self-reinforcing, since the gradient of the residual term with respect to $\hat{t}_j$ itself scales as $1/b_j$; once b_j has inflated, the pressure to correct $\hat{t}_j$ on that candidate weakens further, entrenching poor localization exactly where it is least acceptable.

Precision collapse on easy candidates. In the opposite regime, as $\hat{t}_j \rightarrow t_i$ and the residual shrinks toward zero, $\ell(b_j) \rightarrow \log(2b_j)$, which is unbounded below as $b_j \rightarrow 0$. A candidate that has already learned to localize well can keep reducing its loss indefinitely by driving b_j toward zero, independent of any further gain in accuracy — a numerically unstable, degenerate direction that can dominate the loss landscape and starve gradient signal from harder candidates elsewhere in the batch.

4.1.3 Mitigations

Three changes address these failure modes without abandoning per-candidate precision where it is wanted. First, bound b_j away from zero and infinity by construction rather than by penalty, reusing the device already used for monotonicity in Section 3:

$$b_j(\theta, x) = b_{\min} + \text{softplus}(u_j(\theta, x)),$$

for a fixed floor $b_{\min} > 0$ and a raw head output u_j ; this rules out collapse by construction, the same way (1) rules out crossing by construction, rather than by hoping a penalty term suppresses it in practice. Second, regularize the remaining freedom with a prior: placing a Gamma prior on the precision $1/b_j$ — the natural conjugate prior for a Laplace scale — and optimizing the resulting maximum a posteriori objective adds a term pulling b_j toward a data-informed default, damping runaway inflation on hard candidates rather than only bounding it. Third, decouple the two gradient roles b_j plays: use a stop-gradient on b_j when computing $\partial\mathcal{L}/\partial\hat{t}_j$, so that a single step cannot simultaneously widen b_j and leave $\hat{t}_j$ unimproved; b_j is then updated only through its own, separate loss term, breaking the joint-optimization feedback loop that makes inflation attractive in the first place. A short warm-up phase training with a fixed global b (as in equation (5) above) before unfreezing any per-candidate head is a cheap complement to all three, since it forces accurate localization to be learned before the network has an uncertainty channel available to substitute for it.

A risk specific to this architecture, beyond the general heteroscedastic-regression pathology above, is that $\mathcal{L}_{\text{match}}$ is not only a loss term but also the cost driving the subset-selection problem (4) itself, computed before training loss and gradients are ever formed. If b_j is allowed to inflate, the dynamic program of Section 5 minimizes an already-distorted cost, and can select a worse-fitting $\hat{\sigma}$ precisely because its inflated-precision-adjusted apparent cost looks artificially cheap — corrupting the correspondence, not merely the loss value computed after it. For this reason, if per-candidate precision is used at all, we recommend computing $\mathcal{L}_{\text{match}}$ for the purpose of solving (4) using the shared, periodically updated b from (5) even when the training loss (8) afterward uses the per-candidate b_j — insulating the discrete selection step from a pathology that, by the argument above, is confined to the continuous regression head.

4.2 A Known-Meter Spacing Constraint on σ

The constructions in Section 8 (equation (9)) and Section 10.4 (equation (37)), both introduced later in this document, use a known meter length L , but neither uses it to influence which σ is

selected: Section 8’s use is confined to imputing a missing class label after $\hat{\sigma}$ is already fixed, and Section 10.4’s regularizer is added to the training loss only after $\hat{\sigma}$ is fixed as well. Here we incorporate the same known- L information directly into the cost that Section 5’s dynamic program minimizes, so that the spacing constraint can affect the selection itself; we anticipate the local-beat-period estimate $\bar{\Delta}(\theta, x)$ defined precisely in Section 10.4 rather than redefine it here.

Assume full annotation — both class and time are known for every y_i , unlike Section 8’s setting — and a known, constant meter L . Let $i_1 < i_2 < \dots < i_K \subset \{1, \dots, M\}$ be the ground-truth indices with $c_{i_k} = \text{DB}$; by definition of a constant meter, $i_{k+1} - i_k = L$ for every consecutive pair. We augment the per-pair cost with an additional term, active only when $i = i_{k+1}$ for some k , penalizing deviation of the predicted downbeat-to-downbeat gap from L times the estimated local beat period $\bar{\Delta}(\theta, x)$ defined in equation (36):

$$\mathcal{L}_{\text{meter}}(\sigma(i_k), \sigma(i_{k+1})) = \mu \left(\hat{t}_{\sigma(i_{k+1})}(\theta, x) - \hat{t}_{\sigma(i_k)}(\theta, x) - L \bar{\Delta}(\theta, x) \right)^2, \quad \mu \geq 0. \quad (6)$$

Unlike every per-pair cost used elsewhere in this document, (6) depends on two matched positions jointly, $\sigma(i_k)$ and $\sigma(i_{k+1})$, rather than on $\sigma(i)$ in isolation. This breaks the first-order Markovian structure the recursion (7) relies on: $D[i, j]$ as defined depends on the past only through $D[i - 1, j - 1]$, with no memory of which candidate was matched at any earlier index, whereas (6) needs to know $\sigma(i_k)$ specifically at the later step $i = i_{k+1} = i_k + L$.

An exact but more expensive augmented recursion. The dependency can be restored by carrying the identity of the most recent downbeat’s matched candidate as extra state. Define, for $0 \leq i \leq M$, $0 \leq j \leq N$, and $0 \leq j^* \leq N$,

$D[i, j, j^*] = \text{minimum cost of an order-preserving injection from } \{1, \dots, i\} \text{ into } \{1, \dots, j\} \text{ whose most recent downbeat is } j^*$

with the recursion following (7) unchanged at every non-downbeat index, and, at $i = i_{k+1}$, adding $\mathcal{L}_{\text{meter}}(j^*, j)$ to the branch that sets $\sigma(i_{k+1}) = j$, using whichever j^* that branch’s history carries. This computes the exact minimizer of the augmented cost, but the extra state dimension raises complexity: since j^* ranges over N values but only needs to be tracked between consecutive downbeat indices (there is nothing to remember it against in between), the cost is $O(N^2)$ per inter-downbeat segment rather than $O(N)$, giving $O(N^2 M / L)$ overall rather than Section 5’s $O(NM)$ — a real, non-trivial increase, not a constant-factor one, and one that grows as the meter L shrinks.

Where this sits relative to the two existing constructions. Equation (6) is a strictly more invasive use of known meter than either of the two already in the document: Section 10.4’s regularizer accepts whatever $\hat{\sigma}$ the unconstrained DP already produced and only shapes the gradient afterward, at $O(NM)$ total cost; Section 8’s phase marginalization uses L to resolve a missing label without touching the selection cost at all. The construction here is the only one of the three where the known meter can change which candidates get selected, at the price of the complexity increase above. Which of the three is worth the cost is an empirical question this document does not resolve; we record all three because they trade off exactness of the meter constraint against where in the pipeline it is applied, not because one is derived from another.

5 Solving the Subset-Selection Problem by Dynamic Programming

Equation (7)’s two branches are exactly the two ways of taking a single step through the (j, i) grid below, where grid column $j \in \{0, \dots, N\}$ counts how many candidates have been considered so far

and the candidate newly considered at column j is candidate $j - 1$ under the 0-indexed labeling $j = 0, \dots, N - 1$ of Section 3: a rightward step from $(j-1, i)$ to (j, i) when candidate $j - 1$ is skipped, or a diagonal step from $(j-1, i-1)$ to (j, i) when candidate $j - 1$ is matched to event i — never straight up, since i can only advance alongside j . A path built from these two step types, starting at $(0, 0)$ and ending at (N, M) , is what $\sigma \in \mathfrak{D}_{M,N}$ looks like drawn as a staircase; Figure 1 shows the one such path corresponding to a concrete $\hat{\sigma} = (0, 2, 4)$ for $M=3$, $N=5$ — candidates 0, 2, and 4 matched to events 1, 2, and 3, candidates 1 and 3 skipped. $D[i, j]$ is the minimum cost, summed over $\mathcal{L}'_{\text{match}}$ along diagonal steps, of any staircase reaching (j, i) ; $D[M, N]$ is the minimum over every staircase reaching the far corner, and $\text{choice}[i, j]$, recorded during the forward pass, marks which of the two step types the winning staircase used to arrive there.

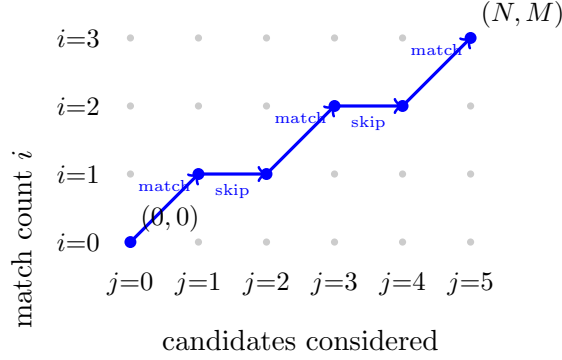


Figure 1: The staircase corresponding to $\hat{\sigma} = (0, 2, 4)$ for $M = 3$, $N = 5$ (candidates labeled $0, \dots, 4$). Candidate position j is the latent axis — each candidate’s own role, matched or background, is what this recursion infers, never given; the state value i indexes the observed ground truth y_i . Both diagonal (match) and horizontal (skip) steps are simultaneously a state transition and an emission, not one or the other: a match advances the state to i while scoring candidate $j-1$ against event i ; a skip leaves the state at i while scoring candidate $j-1$ against background. The path runs from $(0, 0)$ to (N, M) by construction.

The staircase’s own structure is what makes gap invariance hold, not something separate from it. In the figure, the row index i changes only on a diagonal step; every horizontal step moves rightward while i stays exactly where it was. Tracing the example: $(0, 0) \rightarrow (1, 1)$ is a match, $i : 0 \rightarrow 1$, candidate 0 becomes event 1; $(1, 1) \rightarrow (2, 1)$ is a skip, i unchanged, candidate 1 left unmatched; $(2, 1) \rightarrow (3, 2)$ is a match, $i : 1 \rightarrow 2$, candidate 2 becomes event 2; $(3, 2) \rightarrow (4, 2)$ is a skip, candidate 3 unmatched; $(4, 2) \rightarrow (5, 3)$ is a match, $i : 2 \rightarrow 3$, candidate 4 becomes event 3. Because a skip step never touches i , and Section 8’s own phase formula $\phi_i = (\phi_0 + i - 1) \bmod L$ depends on i alone, no skip step — regardless of how many occur in a row — can affect a phase computed this way; only match steps can, and each does so by exactly one. The staircase’s row-versus-column distinction and the gap-invariance property of Section 8 are the same fact seen from two sides, not two facts requiring separate justification.

The cost of each transition type. Every step of the staircase, skip or match alike, is both a state transition and an emission simultaneously, not one or the other: it decides candidate $j-1$ ’s latent role (whether the state advances) while scoring that candidate’s own prediction (against event i if it does, against background if it does not) — the two step types differ only in what gets scored and at what cost, not in whether scoring happens at all. A diagonal (match) step

from $(j-1, i-1)$ to (j, i) incurs $\mathcal{L}_{\text{match}}(y_i, \hat{y}_{j-1})$ — candidate $j-1$ is matched to event i , and the full classification-plus-timing cost of eq. (3) is paid. A horizontal (skip) step from $(j-1, i)$ to (j, i) incurs cost 0: candidate $j-1$ is scored against background, and at this point in the derivation that emission is charged nothing. Two refinements appear later, each affecting only the match branch, never the skip branch. Section 8.4 corrects $\mathcal{L}_{\text{match}}$ to $\mathcal{L}'_{\text{match}} := \mathcal{L}_{\text{match}} - \gamma g_j$, so that matching candidate $j-1$ is properly charged the background-penalty opportunity cost it forgoes by not remaining unmatched — the skip branch’s own 0 cost is untouched, since forgoing nothing costs nothing. Section 8.3 further conditions the match cost on a phase hypothesis, $\mathcal{L}^p_{\text{match}}(y_i, \hat{y}_{j-1})$, using $\phi_{j-1} = (p + i - 1) \bmod L$ in place of the observed c_i — and the skip branch’s cost remains exactly 0 here too, for precisely the structural reason given above: a skip step never advances i , so it has no phase to condition on in the first place, with or without a correction term.

Why skip’s cost is exactly 0, not merely small. Two justifications hold at once, one practical and one principled. Practically, any positive skip cost would bias $\hat{\sigma}$ toward over-matching — pulling the optimum toward absorbing as many candidates as possible purely to avoid repeated penalties, fighting directly against the architecture’s own deliberate overgeneration ($N \gg M$). Principled: $\mathcal{L}_{\text{match}}(y_i, \hat{y}_j) := -\log p(y_i | \hat{y}_j, \theta)$ exists because Section 4 states an explicit observation model for what it means to match event i to candidate j ; there is no analogous observation model, within this sub-problem, for what it means for a candidate to be background — that concern is handled separately, by the γ -weighted penalty in the training loss, not folded into the selection cost itself. Skip costing 0 is the honest reflection that this sub-problem, as posed, has nothing to say about an unmatched candidate’s own merit.

One subtlety is worth making explicit, since it changes between the hard and soft treatments of this document. For the hard DP of this section, skip’s *absolute* value does not actually affect $\hat{\sigma}$: every feasible $\sigma \in \mathfrak{D}_{M,N}$ traces a staircase with exactly M diagonal steps and exactly $N - M$ horizontal steps, forced by simple counting (only diagonal steps advance i , and i must reach M). Adding any constant c to every skip step increases every feasible σ ’s total cost by exactly $c(N - M)$ — the same amount for all of them — leaving $\arg \min_{\sigma}$ completely unchanged. In this narrow sense 0 is a free normalization choice here, not a load-bearing one. This stops being true once Section 8.4 replaces $\arg \min$ with $\sum_{\sigma}$, the Gibbs partition function $Z(\theta, x)$: there, the *absolute* potential of each transition genuinely affects the result, and $\phi_{\text{skip}} = e^{-0} = 1$ specifically means a skip step contributes no distortion to the probability — exactly what licenses reading $Z(\theta, x)$ as a genuine marginal likelihood under the stated observation model, rather than a family of distributions differing by an unjustified per-skip reweighting. The value 0 is chosen freely here with that later, principled reuse already in view.

The boundary conditions are $D[0, j] = 0$ for all j (the empty injection costs nothing), and $D[i, 0] = +\infty$ for $i \geq 1$ (no candidates available cannot cover a nonempty domain). The recursion follows by conditioning on whether candidate $j-1$ lies in the image of σ :

$$D[i, j] = \min \left(\underbrace{D[i, j-1]}_{(j-1) \notin \text{im}(\sigma)}, \underbrace{D[i-1, j-1] + \mathcal{L}_{\text{match}}(y_i, \hat{y}_{j-1})}_{\sigma(i)=j-1} \right), \quad i \geq 1, j \geq 1. \quad (7)$$

The optimal cost is $D[M, N]$, and $\hat{\sigma}$ is recovered by backtracking through the table (Algorithm 1’s own **while** loop, following the recorded choice $[i, j]$ at each step). Filling the table requires $O(NM)$ time and $O(NM)$ space (reducible to $O(N)$ space with a rolling array if only the cost, not the argmin, is needed).

σ reindexed as a state-and-observation sequence, decoded by Viterbi — not a Markov chain, and not, strictly, a hidden Markov model. The staircase of Figure 1 is not merely an illustration of σ ; reindexed, it exposes a state-and-observation structure that $D[i, j]$ computes over via a genuinely Viterbi-shaped recursion — though calling any of this “Markov,” or the object it decodes an HMM outright, overclaims two separate things, each worth being precise about rather than glossed together. σ itself has nothing to do with a Markov chain: it is a single, fixed, deterministic combinatorial object — one specific order-preserving injection, already chosen — and Markov structure is a property of a stochastic process, not of one fixed outcome of one. The question “is σ Markov” is not well-posed until a distribution over σ exists at all, which does not happen until Section 8.4’s Gibbs distribution, and even there the state-dependence and normalization a reader would expect from that word are absent, as established below. What is true, and useful, is narrower and purely algorithmic, a fact about the shape of a computation rather than a claim about σ ’s own probabilistic structure: reindexed via the staircase, $D[i, j]$ ’s recursion has exactly the shape of the Viterbi algorithm’s own recursion.

For each candidate index $j = 0, \dots, N$, let I_j be the staircase’s height at column j — the number of ground-truth events matched using candidates $1, \dots, j$ — giving a sequence $I_0, \dots, I_N$ with $I_0 = 0$, $I_N = M$ always. This sequence is σ under a different name: $\sigma(i) = j$ exactly when I steps from $I_{j-1} = i-1$ to $I_j = i$, so σ is precisely the record of which time steps contain a transition and which do not. This correspondence is exact, not approximate: since σ is an order-preserving injection, $I_j - I_{j-1} \in \{0, 1\}$ at every step (injectivity rules out two events matching the same candidate, order-preservation rules out any other increment), giving a genuine $\{0, +1\}$ -valued state sequence over the state space $\{0, \dots, M\}$, with $D[i, j]$ evaluated at state i , column j . Candidate position j is the latent axis here — which role each candidate plays, matched to a specific event or background, is exactly what this recursion infers, never given — while the state value i indexes the observed ground truth y_i , given as data throughout. Every step, skip or match alike, is accordingly both a transition and an emission at once, as the transition-cost paragraph above already makes explicit for both step types, not a transition-only versus emission-only split between them.

$D[i, j] = \min(D[i, j-1], D[i-1, j-1] + \mathcal{L}'_{\text{match}}(y_i, \hat{y}_{j-1}))$ is exactly the Viterbi-shaped recursion for this state sequence: under the Gibbs transform $V[i, j] := e^{-D[i, j]}$, SKIP and MATCH become $\phi_{\text{skip}} = 1$ and $\phi_{\text{match}}(i, j) = e^{-\mathcal{L}'_{\text{match}}(y_i, \hat{y}_{j-1})}$, genuine non-negative potentials, and $D[M, N] = -\log V[M, N]$ is the negative log-weight of the highest-scoring state sequence under them — a max-product computation, structurally identical to Viterbi decoding.

Deriving V ’s max-product recursion from D ’s min-plus one. Write $a := D[i, j-1]$ and $b := D[i-1, j-1] + \mathcal{L}'_{\text{match}}(y_i, \hat{y}_{j-1})$, so $D[i, j] = \min(a, b)$ by definition. For any $a, b \in \mathbb{R}$, $e^{-\min(a, b)} = \max(e^{-a}, e^{-b})$: assuming without loss of generality $a \leq b$, $\min(a, b) = a$ gives $e^{-\min(a, b)} = e^{-a}$, and since $x \mapsto e^{-x}$ is strictly decreasing, $a \leq b$ forces $e^{-a} \geq e^{-b}$, so $\max(e^{-a}, e^{-b}) = e^{-a}$ as well — the two sides agree (the case $b \leq a$ is symmetric). Applying this to $D[i, j] = \min(a, b)$,

$$V[i, j] = e^{-\min(a, b)} = \max(e^{-a}, e^{-b}) = \max\left(e^{-D[i, j-1]}, e^{-D[i-1, j-1]} \cdot e^{-\mathcal{L}'_{\text{match}}(y_i, \hat{y}_{j-1})}\right),$$

using $e^{-(D[i-1, j-1] + \mathcal{L}'_{\text{match}})} = e^{-D[i-1, j-1]} e^{-\mathcal{L}'_{\text{match}}}$ in the second term. Substituting the definitions of $V[i, j-1]$, $V[i-1, j-1]$, and $\phi_{\text{match}}(i, j)$, and using $\phi_{\text{skip}} := 1$ so that $V[i, j-1] = V[i, j-1] \cdot \phi_{\text{skip}}$, this is exactly $V[i, j] = \max(V[i, j-1] \cdot \phi_{\text{skip}}, V[i-1, j-1] \cdot \phi_{\text{match}}(i, j))$, the claimed max-product form. The base cases transform correctly under the same map: $D[0, j] = 0 \Rightarrow V[0, j] = 1$; $D[i, 0] = +\infty$ ($i \geq 1$) $\Rightarrow V[i, 0] = 0$; and $D[M, N] = -\log V[M, N]$ follows by taking $-\log$ of $V[M, N] := e^{-D[M, N]}$ directly.

What keeps this from being a hidden Markov model in the technical sense: two conditions, not one. Two things are required of a genuine HMM transition kernel, and it is worth separating them rather than naming only the first. *Normalization*: $P(\text{skip} \mid \cdot) + P(\text{match} \mid \cdot) = 1$ at every state. *State-dependence*: this probability must be a function of a compact state alone — the same number every time the chain occupies that state, reused across every occasion it does, not recomputed fresh from whatever data happens to be at hand this particular time. ϕ_{skip} and $\phi_{\text{match}}(i, j)$ fail the first outright: $\phi_{\text{skip}} + \phi_{\text{match}}(i, j) = 1 + \phi_{\text{match}}(i, j)$ varies with (i, j) rather than summing to a fixed total.

Normalizing does not, by itself, repair this, and it is worth checking directly rather than asserting it: define $P_{\text{match}}(i, j) := \phi_{\text{match}}(i, j) / (1 + \phi_{\text{match}}(i, j))$ and $P_{\text{skip}}(i, j) := 1 / (1 + \phi_{\text{match}}(i, j))$, which do sum to 1 at every cell. But $\phi_{\text{match}}(i, j) = e^{-\mathcal{L}_{\text{match}}(y_i, \hat{y}_{j-1})}$ is built from the specific ground-truth event y_i and the specific candidate $\hat{y}_{j-1}$, so $P_{\text{match}}(i, j)$ takes a different value at every cell, freshly computed from that cell’s own data — never reused. Two cells representing the identical nominal state (say, $i = 3$, matched so far regardless of which candidates were involved) have unrelated transition probabilities, computed from entirely different data, sharing no parameter. This fails state-dependence regardless of normalization: the transition is a function of raw position and data, not of state, and normalizing the arithmetic at each cell does nothing to make cells representing the same state agree with one another. This is exactly the same gap already established for SubsetSelectDP more generally: a state-and-observation sequence, decoded by a genuinely Viterbi-shaped recursion, over potentials that are real and non-negative but neither normalized nor state-dependent — the precise sense in which the staircase is Viterbi’s object without yet being an HMM’s, and without σ itself ever having been a Markov chain’s object in the first place.

What this costs representationally, and what it does not. Three things follow from lacking the state-dependence property, not all in the same direction. Genuinely lost: an independent, learnable transition prior separate from observation fit — a real HMM can express “matches are generally common here” independent of how well any specific candidate fits, while here $\phi_{\text{match}}(i, j)$ is the observation fit, with nothing else contributing; and a local, reusable distribution to marginalize cell to cell, rather than only the global $P(\sigma)$ established above, forcing every marginal computation through the full table rather than composing from a small, shared piece. Not actually lost, contrary to what the general pattern might suggest: state persistence across a skip run. This matters elsewhere in this document (Section 8’s own phase construction) but not here, because $\phi_i = (\phi_0 + i - 1) \bmod L$ is a closed-form deterministic formula rather than something carried forward step by step — it gets this property for free, by never needing a persistence mechanism at all, not by having a working one. Gained, on the other side of the trade: no separate transition parameters to learn, fit, or risk disagreeing with the observation model — the entire transition behavior is parameter-free, derived directly from the same cost the observation model of Section 4 already justifies.

Is this construction more global, and does that make training more expensive? More global: yes, and this is the same fact already established for the Gibbs distribution above, restated in HMM terms — the only well-defined probability object here is the global $P(\sigma)$ over the entire alignment; there is no local, per-cell distribution analogous to a genuine transition kernel that a forward-backward pass could reuse, only a global object assembled from locally-computed pieces. More expensive to train: no, and this is worth checking rather than assumed, since “richer model, more compute” does not actually hold here. The $O(NM)$ cost comes from the size of the (i, j) grid — sequence length N times state-space size $M+1$ — which a genuine HMM with the identical

sparse, two-outgoing-edge transition structure over the identical state space would incur just as much: filling the grid costs the same whether the per-cell weights are reusable learned parameters or freshly computed from that cell’s own data. Space is likewise unaffected: both constructions admit the same $O(N)$ rolling-array reduction when only the final cost is needed, and both require the full $O(NM)$ table when backtracking or marginals are required, for the same reason a genuine HMM’s forward-backward would. If anything, the missing state-dependence makes training cheaper in one specific sense: there is no separate transition parameter set to fit alongside θ , and hence no risk of a learned transition prior drifting out of step with what the classification and regression heads already believe.

A sharper characterization: emission cost, not transition weight. The roles in this construction are better read the other way around from a naive HMM reading. Take candidate position j as the axis playing the role of time, and $I_j \in \{0, \dots, M\}$ — the staircase’s own height, established above — as the latent state indexed by j : skip leaves $I_j = I_{j-1}$, match increments $I_j = I_{j-1} + 1$. What is scored at a match is not the transition itself but an emission: the observation y_i , $i := I_j$ — event-indexed, and never latent, since the ground truth is given — checked against candidate $j-1$ ’s own prediction $\hat{y}_{j-1}$ via $p(y_i \mid \hat{y}_{j-1}, \theta)$, Section 4’s own observation model. The whole of $\mathcal{L}_{\text{match}}$ is this emission cost, nothing else. This is a stronger claim than the state-dependence failure established above, not merely a restatement of it: a genuine HMM’s match step would carry *two* independently-weighted factors, a transition probability and an emission probability, multiplied. Here the transition factor is entirely absent — skip costs 0, and match’s own transition component, as distinct from its emission, is likewise unweighted — so every unit of cost in this construction is emission, none is transition. The construction is accordingly better placed in a different family than “HMM with an irregular transition weight”: it is a scoring lattice, with the set of legal moves through the (i, j) grid fixed deterministically in advance and all learned weight carried entirely by local emission scores — precisely the structure CTC and DTW below share, and a sharper version of the connection to them than shared asymptotic complexity alone would suggest.

Algorithm 1 SubsetSelectDP — order-constrained subset selection via dynamic programming

Require: ground truth $y_1, \dots, y_M$; candidates $\hat{y}_0, \dots, \hat{y}_{N-1}$; cost function $\mathcal{L}_{\text{match}}$

```
1: Initialize  $D[0, j] \leftarrow 0$  for  $j = 0, \dots, N$ ;  $D[i, 0] \leftarrow +\infty$  for  $i = 1, \dots, M$ .  $\triangleright D[\cdot, j]$  counts
   candidates considered; column  $j$ 's own candidate is  $\hat{y}_{j-1}$ 
2: for  $i = 1, \dots, M$  do
3:   for  $j = 1, \dots, N$  do
4:      $c_{\text{skip}} \leftarrow D[i, j-1]$ ;  $c_{\text{match}} \leftarrow D[i-1, j-1] + \mathcal{L}_{\text{match}}(y_i, \hat{y}_{j-1})$ 
5:     if  $c_{\text{match}} < c_{\text{skip}}$  then
6:        $D[i, j] \leftarrow c_{\text{match}}$ ;  $\text{choice}[i, j] \leftarrow \text{MATCH}$ 
7:     else
8:        $D[i, j] \leftarrow c_{\text{skip}}$ ;  $\text{choice}[i, j] \leftarrow \text{SKIP}$ 
9:     end if
10:   end for
11: end for
12:  $i \leftarrow M$ ;  $j \leftarrow N$ 
13: while  $i > 0$  do
14:   if  $\text{choice}[i, j] = \text{MATCH}$  then
15:      $\hat{\sigma}(i) \leftarrow j-1$ ;  $i \leftarrow i-1$ ;  $j \leftarrow j-1$   $\triangleright$  candidate  $j-1$ , not  $j$ , is what column  $j$  represents
16:   else
17:      $j \leftarrow j-1$ 
18:   end if
19: end while
20: return  $\hat{\sigma}$ 
```

Candidates not in the image of $\hat{\sigma}$ are, by construction, treated as predicting the background class $\emptyset$.

6 Optimality and Scope of the Order Constraint

Proposition 1 (Exactness of the dynamic program). *The recursion (7) computes $D[M, N]$ equal to the true minimum in (4) over all order-preserving injections $\sigma \in \mathfrak{D}_{M, N}$.*

Proof sketch. By induction on i . The claim holds trivially for $i = 0$. Assume $D[i-1, j']$ is the true minimum cost of an order-preserving injection from $\{1, \dots, i-1\}$ into $\{0, \dots, j'-1\}$ — the first j' candidates, 0-indexed — for every $j' < j$. Any order-preserving injection from $\{1, \dots, i\}$ into $\{0, \dots, j-1\}$ either has $(j-1) \notin \text{im}(\sigma)$, in which case its cost is bounded below by $D[i, j-1]$, or has $\sigma(i) = j-1$, in which case its restriction to $\{1, \dots, i-1\}$ is an order-preserving injection into $\{0, \dots, j-2\}$, whose cost is bounded below by $D[i-1, j-1]$ by the inductive hypothesis, plus the cost $\mathcal{L}_{\text{match}}(y_i, \hat{y}_{j-1})$ of setting $\sigma(i) = j-1$. The recursion takes the minimum of these two cases, so $D[i, j]$ is the true minimum over injections into $\{0, \dots, j-1\}$. $\square$

This establishes that the dynamic program solves (4) exactly, with no approximation, for whatever cost function $\mathcal{L}_{\text{match}}$ is used. What remains open is a separate question: whether requiring the correspondence to be order-respecting in the first place ever discards a lower-cost correspondence that violates order.

Remark 1. *If the classification term in (3) were removed, leaving only the pure time term $|t_i - \hat{t}_j|$, a standard exchange argument for sorted real-valued sequences shows that no order-violating*

correspondence can achieve a strictly lower total cost than the best order-respecting one: swapping a crossing pair of assignments never increases the total absolute error when both underlying sequences are sorted. Under the full cost (3), this guarantee does not automatically carry over, because an order-violating correspondence could in principle achieve a lower total classification cost than any order-respecting one, for an untrained or unusually structured network. Equation (4) would then no longer be guaranteed to reach the same value as an unconstrained correspondence search.

We nonetheless adopt the order constraint, for three reasons stated as assumptions rather than proven facts. First, the architectural reparameterization of Section 3 already forbids the network from producing a crossing hypothesis, so an order-respecting correspondence is the only kind consistent with what the network can express in the first place; an unconstrained search would be searching over correspondences the model cannot realize. Second, for a beat-tracking signal, a ground-truth-to-prediction pairing that violates temporal order is physically nonsensical regardless of classification confidence, so such pairings should not be rewarded even where they are formally cheaper. Third, the $O(NM)$ complexity of the dynamic program is preferable to the higher complexity of an unconstrained correspondence search at training time. The second point in particular is a modeling judgment, not a mathematical necessity, and should be stated as such if this formulation is used in downstream work.

7 Training Loss For Fully-Labeled Datasets

$\hat{\sigma}$ from equation (4) is the exact minimizer of $\sum_i \mathcal{L}_{\text{match}}(y_i, \hat{y}_{\sigma(i)})$ over $\mathfrak{D}_{M,N}$; because Section 4 derives $\mathcal{L}_{\text{match}}(y_i, \hat{y}_j)$ as $-\log p(y_i | \hat{y}_j, \theta)$ under observation model (2), this minimization is, by ordinary Bayes’ rule under a uniform prior over $\mathfrak{D}_{M,N}$, exactly the maximization of $P(\sigma | y_{1:M}, \theta, x)$: $\hat{\sigma}$ is the posterior mode of the latent function σ , not a different or unrelated object. Once obtained, $\hat{\sigma}$ is held fixed — detached from the computational graph, receiving no gradient — and the differentiable training loss for a single fragment is

$$\mathcal{L}(\theta) = \sum_{i=1}^M \left[\omega_{c_i} (-\log \hat{p}_{\hat{\sigma}(i)}(c_i | x; \theta)) + \lambda_{L1} |t_i - \hat{t}_{\hat{\sigma}(i)}(\theta, x)| \right] + \gamma \sum_{j \notin \text{im}(\hat{\sigma})} -\log \hat{p}_j(\emptyset | x; \theta), \quad (8)$$

where $\gamma \in (0, 1]$ down-weights the background-class term to counteract the class imbalance created by the $N - M$ inactive candidates, and $\omega_c > 0$ is a fixed, class-specific weight on the matched classification term, with $\omega_B = 1$ as the reference scale. Setting $\omega_{DB} = 1$ recovers unweighted per-class treatment; Section 10 discusses why $\omega_{DB} > 1$ is warranted. With the correction to (3) in Section 4, the selection cost and the training loss now use the class term in the same form, $-\log \hat{p}(\cdot | x; \theta)$; the training loss departs from a pure negative log-likelihood only in the class-specific weight ω_{c_i} and the background down-weight γ , both introduced as explicit importance-reweighting choices to counteract class imbalance (Section 10), not as consequences of the observation model in (2).

Because $\hat{\sigma}(\theta, x)$ is recomputed by dynamic programming at every forward pass, this training procedure alternates a non-differentiable selection step, evaluated under the current parameters, with a differentiable parameter update under $\hat{\sigma}$ held fixed. This is, correctly, an instance of hard-EM: the selection step is an E-step that collapses the posterior $P(\sigma | y_{1:M}, \theta, x)$ to a point mass at its mode $\hat{\sigma}$ — the same restricted-family construction known elsewhere as Classification EM (Celeux & Govaert, 1992) or Viterbi training, and equivalently describable as an instance of Dirac EM, the posterior collapsed to $\delta_{\hat{\sigma}}$ rather than kept in full — and the parameter update is the corresponding M-step, evaluating the expected complete-data objective under that point mass,

which reduces exactly to $\hat{\sigma}$ held fixed while θ is updated. σ 's status as a genuinely latent function, established in Section 4, is what makes this description correct rather than merely evocative: there is a real posterior being collapsed, not an analogy to one.

8 Training Loss For Beat-Only Labeled Datasets

The single-event marginal, and why it is insufficient on its own. For a matched candidate $j = \hat{\sigma}(i)$ drawn from a beat-only-labeled track, all that is known about the true class c_i is that it lies in $\{B, DB\}$ — the annotation records only $c_i = B_*$, not which. Marginalizing the classification term over this superset rather than committing to either label,

$$\ell_i(\theta) = -\log \left(\hat{p}_{\hat{\sigma}(i)}(B \mid x; \theta) + \hat{p}_{\hat{\sigma}(i)}(DB \mid x; \theta) \right) = -\log \left(1 - \hat{p}_{\hat{\sigma}(i)}(\emptyset \mid x; \theta) \right), \quad (9)$$

correctly supervises candidate $\hat{\sigma}(i)$ toward being recognized as *some* beat, but supplies no signal at all toward distinguishing which kind: its gradient with respect to any redistribution of probability between B and DB, holding their sum fixed, is exactly zero, since both classes enter (9) only through their sum. This is a structural fact about what a single event's own marginal can express, not a defect of this particular algebraic form; no reformulation of $\ell_i(\theta)$ built from candidate $\hat{\sigma}(i)$'s own prediction alone escapes it, since the observation $c_i \in \{B, DB\}$ itself resolves nothing further. What follows constructs a loss that pools evidence across the M matched events of the fragment, coupled by the meter, rather than attempting to extract more from any single event's own prediction than the annotation actually supports.

The training objective: marginal log-likelihood over the shared phase latent. Fix $\phi_0 \in \{0, \dots, L-1\}$ as the single latent this fragment's beat-only annotation leaves unresolved — the phase of the first matched event, from which every other matched event's phase follows deterministically, $\phi_{i+1} = (\phi_i + 1) \bmod L$. This increment is indexed by i , the count of *matched* events, never by candidate position: the phase assigned to matched event $i+1$ differs from event i 's by exactly one regardless of how many unmatched candidates sit between $\hat{\sigma}(i)$ and $\hat{\sigma}(i+1)$, since an unmatched candidate is simply not counted. This is the concrete mechanism behind Section 1's own framing of this whole problem as latent order-preserving alignment: an unmatched candidate here plays the same role CTC's own blank symbol plays in its alignment, contributing nothing to the count regardless of how many intervene between two real events.

L itself is treated as known, fixed external metadata throughout this derivation and Sections 8.1–8.5, exactly as Section 4.2 already treats it for the fully-labeled case — a genuine simplifying assumption, not a claim that L is somehow not latent in general: L is every bit as much an unobserved quantity as σ and ϕ_0 whenever it is not reliably annotated, and it is treated as such, marginalized jointly with (σ, ϕ_0) , only once Section 8.6 removes this assumption.

The phase-valued classification probability, in place of $c(p, i)$. Rather than name a separate function $c(p, i)$ mapping a hypothesis to a class label, work directly with $\phi_{\hat{\sigma}(i)}$ — event i 's own matched candidate's phase, exactly the quantity established above to be equivalent to that candidate's class. For any candidate j and hypothesized phase value $q \in \{0, \dots, L-1\}$, extend the classification head's own output to accept a phase-valued query:

$$\hat{p}_j(\phi_j = q \mid x; \theta) := \begin{cases} \hat{p}_j(DB \mid x; \theta), & q = 0, \\ \hat{p}_j(B \mid x; \theta), & q \neq 0, \end{cases}$$

collapsing every nonzero phase value onto the same B probability, since the classification head itself only ever distinguishes downbeat from non-downbeat, never one non-downbeat phase from another.

Under the fragment-wide hypothesis $\phi_0 = p$, event i 's own matched candidate has hypothesized phase $\phi_{\hat{\sigma}(i)} = (p + i - 1) \bmod L$, by the same deterministic propagation rule established above; querying $\hat{p}_{\hat{\sigma}(i)}(\phi_{\hat{\sigma}(i)} = (p + i - 1) \bmod L \mid x; \theta)$ is therefore exactly the class lookup this section needs, expressed directly in phase language rather than through an intermediate class-valued function.

The full log-joint-likelihood, including timing and the background penalty. Each matched event's observation is $y_i = (c_i, t_i)$, not c_i alone, and the fragment's full training-time objective also includes a term for every candidate $\hat{\sigma}$ leaves unmatched. The per-event likelihood given a match to $\hat{\sigma}(i)$, and a hypothesized phase $\phi_0 = p$, is the product of two conditionally independent channels — a categorical term for class, expressed via $\phi_{\hat{\sigma}(i)}$, and a Laplace term for timing:

$$P(y_i \mid \hat{\sigma}, \phi_0 = p; \theta) = \hat{p}_{\hat{\sigma}(i)}\left(\phi_{\hat{\sigma}(i)} = (p + i - 1) \bmod L \mid x; \theta\right) \cdot \frac{1}{2b} \exp\left(-\frac{|t_i - \hat{t}_{\hat{\sigma}(i)}(\theta, x)|}{b}\right).$$

This is built from candidate $\hat{\sigma}(i)$'s own hypothesized phase, not the observed c_i directly, and deliberately so: for a beat-only event $c_i = B_*$ carries no entry in the three-way distribution $\{\text{DB}, B, \emptyset\}$ at all, so Section 8.4's own per-event likelihood, $P(y_i \mid \hat{y}_{\hat{\sigma}(i)}, \theta) = \hat{p}_{\hat{\sigma}(i)}(c_i \mid \cdot) \cdot (\dots)$, built from c_i directly, is simply not evaluable here — recalling it as the basis for this fragment's objective would be recalling something undefined for the very data this section addresses. $\phi_{\hat{\sigma}(i)}$'s own hypothesized value is what beat-only training actually has available to score a class against, and it is what $P(y_i \mid \hat{\sigma}, \phi_0 = p; \theta)$ above correctly uses. Write $P(y_{1:M}, \phi_0 = p \mid x; \theta) := \prod_{i=1}^M P(y_i \mid \hat{\sigma}, \phi_0 = p; \theta)$ for the resulting joint likelihood of all M matched events under one phase hypothesis.

Two things in this construction do not depend on p at all. The timing factor is built entirely from the observed time t_i and the matched candidate's own predicted time $\hat{t}_{\hat{\sigma}(i)}(\theta, x)$, neither of which changes under different phase hypotheses. And the background term, $\hat{p}_j(\emptyset \mid x; \theta)$ for each unmatched $j \notin \text{im}(\hat{\sigma})$, concerns a candidate that plays no role in the matched events' periodic pattern at all — ϕ_0 describes only how the M matched events' own downbeat-versus-beat identity is arranged, and an unmatched candidate's own background probability has nothing to do with that arrangement. Only the classification factor, $\hat{p}_{\hat{\sigma}(i)}(\phi_{\hat{\sigma}(i)} = (p + i - 1) \bmod L \mid \cdot)$, genuinely depends on which hypothesis p is considered.

Combining the marginalized per-event likelihood with the background term — exactly the same background factor Section 8.4's own $P_{\hat{\sigma}}(\theta, x)$ uses, now paired with a per-event likelihood that is actually well-defined for this data — gives the fragment's full joint quantity:

$$P_{\hat{\sigma}}^{\text{beat}}(\theta, x) := \left(\sum_{p=0}^{L-1} P(y_{1:M}, \phi_0 = p \mid x; \theta) \right) \cdot \prod_{j \notin \text{im}(\hat{\sigma})} \hat{p}_j(\emptyset \mid x; \theta)^\gamma. \quad (10)$$

The full log-joint-likelihood for the fragment is therefore

$$\begin{aligned} \log P_{\hat{\sigma}}^{\text{beat}}(\theta, x) &= \log \sum_{p=0}^{L-1} P(y_{1:M}, \phi_0 = p \mid x; \theta) + \gamma \sum_{j \notin \text{im}(\hat{\sigma})} \log \hat{p}_j(\emptyset \mid x; \theta) \\ &= \underbrace{\sum_{i=1}^M \left[-\log(2b) - \frac{|t_i - \hat{t}_{\hat{\sigma}(i)}(\theta, x)|}{b} \right]}_{\text{timing term, no } p\text{-dependence}} + \underbrace{\gamma \sum_{j \notin \text{im}(\hat{\sigma})} \log \hat{p}_j(\emptyset \mid x; \theta)}_{\text{background term, no } p\text{-dependence}} \\ &\quad + \log \sum_{p=0}^{L-1} \prod_{i=1}^M \hat{p}_{\hat{\sigma}(i)}\left(\phi_{\hat{\sigma}(i)} = (p + i - 1) \bmod L \mid x; \theta\right), \end{aligned} \quad (11)$$

the second equality obtained by expanding $P(y_{1:M}, \phi_0 = p \mid x; \theta)$'s own definition, noting the timing factor's product over i does not depend on p and so pulls entirely outside $\sum_p$, and taking logarithms. Both the timing and background terms are constants with respect to p , so each pulls outside $\sum_p$ before the logarithm is taken. Neither requires any E-step or M-step machinery, precisely because neither is affected by the phase uncertainty at all: the timing term, once the constant $-\log(2b)$ is dropped, is exactly $-\lambda_{L1} \sum_i |t_i - \hat{t}_{\hat{\sigma}(i)}(\theta, x)|$, the same regression term eq. (8) already includes for every matched event, beat-only or not; the background term is exactly eq. (8)'s own γ -weighted unmatched-candidate penalty, unchanged. What remains — the last term of eq. (11) — is the only piece genuinely affected by the unresolved phase: the classification likelihood of the M matched events, coupled through the shared, genuinely uncertain ϕ_0 . This is exactly what this section's E-step and M-step are built to handle, and it is the only part of the fragment's full objective this derivation needs to concern itself with — the timing and background terms are already correctly present elsewhere in the total loss and require nothing further here.

Why this needs care to differentiate usefully. Eq. (11) is a log of a sum, and while directly differentiating it is tractable here (since L is small), doing so gives no natural decomposition into a per-event loss — no analogue of the training loop's own per-matched-event structure (eq. (8) sums a term over each i separately). What follows constructs exactly such a decomposition, and shows it optimizes eq. (11) correctly rather than some other, easier quantity.

A lower bound, for any distribution over the latent. For any distribution $q(p)$ over $\{0, \dots, L-1\}$ with $q(p) > 0$ wherever the corresponding term above is nonzero,

$$\log P(y_{1:M} \mid x; \theta) = \log \sum_p q(p) \frac{P(y_{1:M}, \phi_0 = p \mid x; \theta)}{q(p)} \geq \sum_p q(p) \log \frac{P(y_{1:M}, \phi_0 = p \mid x; \theta)}{q(p)} =: \mathcal{F}(q, \theta),$$

by Jensen's inequality, since $\log$ is concave. $\mathcal{F}(q, \theta)$ is a valid lower bound on the true objective for *every* choice of q , not just a specific one — this holds before any commitment about how q is chosen.

E-step: the tightest possible choice of q . Jensen's inequality is an equality exactly when its argument, $P(y_{1:M}, \phi_0 = p \mid x; \theta)/q(p)$, is constant across every p with $q(p) > 0$ — which, combined with $\sum_p q(p) = 1$, forces $q(p) \propto P(y_{1:M}, \phi_0 = p \mid x; \theta)$, i.e.

$$q(p) = P(\phi_0 = p \mid y_{1:M}, x; \theta),$$

the exact posterior. So at the current parameters θ_{old} , setting

$$\pi_p := P(\phi_0 = p \mid y_{1:M}, x; \theta_{\text{old}}) = \frac{\prod_i \hat{p}_{\hat{\sigma}(i)}(\phi_{\hat{\sigma}(i)} = (p + i - 1) \bmod L \mid x; \theta_{\text{old}})}{\sum_{p'} \prod_i \hat{p}_{\hat{\sigma}(i)}(\phi_{\hat{\sigma}(i)} = (p' + i - 1) \bmod L \mid x; \theta_{\text{old}})} \quad (12)$$

makes the bound exactly tight at $\theta = \theta_{\text{old}}$: $\mathcal{F}(\pi, \theta_{\text{old}}) = \log P(y_{1:M} \mid x; \theta_{\text{old}})$. This is the E-step, and it is *derived* here — the unique choice of q making Jensen's inequality tight — not introduced as a separate rule alongside the M-step.

M-step: maximize the tight bound, not the true objective directly. With $q = \pi$ fixed at its θ_{old} value,

$$\mathcal{F}(\pi, \theta) = \sum_p \pi_p \log P(y_{1:M}, \phi_0 = p \mid x; \theta) - \sum_p \pi_p \log \pi_p,$$

and the second term does not depend on θ at all, since π is held fixed. So maximizing $\mathcal{F}(\pi, \theta)$ over θ is exactly equivalent to maximizing

$$Q(\theta; \theta_{\text{old}}) := \sum_p \pi_p \log P(y_{1:M}, \phi_0 = p \mid x; \theta) = \sum_p \pi_p \sum_{i=1}^M \log \hat{p}_{\hat{\sigma}(i)}(\phi_{\hat{\sigma}(i)} = (p + i - 1) \bmod L \mid x; \theta). \quad (13)$$

This is the M-step, likewise derived rather than asserted: it is what maximizing the E-step's own tight bound reduces to, once the θ -independent entropy term is dropped.

Event i 's own phase is directly meaningful, independent of any regrouping: $\phi_i = 0$ means, by definition, that event i is a downbeat, and $\phi_i = 0$ exactly when the hypothesis $\phi_0 = p$ satisfies $(p + i - 1) \bmod L = 0$. This condition determines p uniquely: for fixed i and L , exactly one $p \in \{0, \dots, L - 1\}$ satisfies $(p + i - 1) \bmod L = 0$, namely

$$p_i^* := (1 - i) \bmod L,$$

so $\{p : (p + i - 1) \bmod L = 0\} = \{p_i^*\}$ is a singleton, not a general subset — each event has exactly one phase hypothesis under which it would be a downbeat, never several, verified directly: $(p_i^* + i - 1) \bmod L = ((1 - i) + i - 1) \bmod L = 0 \bmod L = 0$. So event i 's marginal downbeat probability, under the current posterior π over ϕ_0 , is a direct lookup rather than a genuine sum:

$$r_i(\theta_{\text{old}}) := P(\phi_i = 0 \mid x; \theta_{\text{old}}) = \pi_{p_i^*}, \quad (14)$$

following directly from π_p 's own meaning as $P(\phi_0 = p \mid y_{1:M}, x; \theta_{\text{old}})$ together with ϕ_i 's deterministic dependence on ϕ_0 — not from any regrouping of eq. (13). Substituting this definition into eq. (13)'s own double sum, grouped by event rather than by phase hypothesis, gives exactly

$$Q(\theta; \theta_{\text{old}}) = \sum_{i=1}^M \left[r_i(\theta_{\text{old}}) \log \hat{p}_{\hat{\sigma}(i)}(\text{DB} \mid x; \theta) + (1 - r_i(\theta_{\text{old}})) \log \hat{p}_{\hat{\sigma}(i)}(\text{B} \mid x; \theta) \right], \quad (15)$$

so maximizing eq. (13) is exactly minimizing $-\sum_i \ell_i(\theta)$ with $\ell_i(\theta) := -r_i(\theta_{\text{old}}) \log \hat{p}_{\hat{\sigma}(i)}(\text{DB} \mid x; \theta) - (1 - r_i(\theta_{\text{old}})) \log \hat{p}_{\hat{\sigma}(i)}(\text{B} \mid x; \theta)$ — the training loop's own per-event loss, each event weighted by its own r_i , but with every r_i genuinely coupled to every other matched event's prediction through the shared posterior π , not computed from any single event's own prediction in isolation. Comparing (15) to (9) makes precise what pooling across the fragment buys: $\ell_i(\theta)$ here has a genuinely nonzero gradient along a B-versus-DB redistribution direction for candidate $\hat{\sigma}(i)$, whenever $r_i(\theta_{\text{old}})$ was itself shaped by some *other* matched event's prediction through π — precisely the coupling (9) lacked by never looking beyond event i 's own prediction at all.

Guarantees this construction provides. *Monotonicity:* $\mathcal{F}(\pi, \theta) \leq \log P(y_{1:M} \mid x; \theta)$ for every θ (Jensen, general $q = \pi$), with equality at θ_{old} specifically (by the E-step's construction), so

$$\log P(y_{1:M} \mid x; \theta_{\text{new}}) \geq \mathcal{F}(\pi, \theta_{\text{new}}) \geq \mathcal{F}(\pi, \theta_{\text{old}}) = \log P(y_{1:M} \mid x; \theta_{\text{old}})$$

whenever the M-step does not decrease $\mathcal{F}$ — the true objective can never get worse from one EM iteration to the next. *Fisher's identity:* differentiating $\mathcal{F}(\pi, \theta)$ at $\theta = \theta_{\text{old}}$, the entropy term drops out as before, giving $\nabla_{\theta} \mathcal{F}(\pi, \theta)|_{\theta_{\text{old}}} = \nabla_{\theta} Q(\theta; \theta_{\text{old}})|_{\theta_{\text{old}}}$; combined with tightness at θ_{old} , this equals $\nabla_{\theta} \log P(y_{1:M} \mid x; \theta)|_{\theta_{\text{old}}}$, in sharp contrast to the single-event loss (9), whose gradient along every B-versus-DB redistribution direction vanishes identically.

Iteration. The next training step recomputes π , hence every r_i , fresh from the updated θ — exactly Algorithm 3’s own structure, eq. (14) recomputed every forward pass — rather than caching a stale E-step, precisely what this derivation requires for the monotonicity guarantee to keep holding at each successive iteration.

8.1 Extending to Mixed Meter Within a Fragment

Everything above assumes a single, known meter L for the whole fragment — the same scoped assumption stated at the opening of Section 8, not repeated here in full: L enters eq. (11) only as the upper limit of the sum over p and as a fixed constant inside $\phi_{\hat{\sigma}(i)}$ ’s own hypothesized value, $(p + i - 1) \bmod L$, never as an argument of $P(\cdot \mid \cdot)$ requiring inference. The one genuine latent throughout the single-meter derivation specifically is ϕ_0 , whose value determines every other matched event’s phase via $\phi_{i+1} = (\phi_i + 1) \bmod L$. This fails exactly where the meter itself changes within a fragment — a bar of 3/4 followed by bars of 4/4, say — since a phase count carried across such a change is not a meaningful quantity under either meter on its own: there is no natural way to continue a count taken modulo 3 into one taken modulo 4.

Assume, as Section 4.2 already does for the fully-labeled case, that the meter and its change-points are known, external metadata rather than something inferred from the audio: the fragment’s matched events split into κ segments at known indices $1 = i_1 < i_2 < \dots < i_\kappa \leq M$ (writing $i_{\kappa+1} := M + 1$), segment k consisting of events $i_k, \dots, i_{k+1} - 1$, each with its own known meter L_k . Segment k gets its own phase-origin latent $\phi_{i_k} \in \{0, \dots, L_k - 1\}$, playing exactly the role ϕ_0 played for the whole fragment before, with $\phi_{i+1} = (\phi_i + 1) \bmod L_k$ propagating within the segment and resetting at each new segment’s own i_k . The class lookup generalizes to be segment-aware: for event i in segment k and a hypothesized $\phi_{i_k} = p \in \{0, \dots, L_k - 1\}$,

$$c_k(p, i) := \begin{cases} \text{DB}, & (p + i - i_k) \bmod L_k = 0, \\ \text{B}, & \text{otherwise.} \end{cases}$$

The joint marginal factorizes across segments — derived, not assumed. The claim needing justification is that segments’ phases carry no information about one another, so the fragment’s full beat-only classification likelihood, marginalizing jointly over $(\phi_{i_1}, \dots, \phi_{i_\kappa})$, reduces to a product of κ independent single-segment marginals rather than one κ -fold sum. This follows directly once the joint likelihood is written out: each event i belongs to exactly one segment k , so $\hat{p}_{\hat{\sigma}(i)}(c_k(p_k, i) \mid x; \theta)$ depends only on that segment’s own hypothesis p_k , never on any other segment’s. The product over all M matched events therefore already factors by segment before any sum is taken,

$$\prod_{i=1}^M \hat{p}_{\hat{\sigma}(i)}(c(p_{k(i)}, i) \mid x; \theta) = \prod_{k=1}^{\kappa} \prod_{i: i_k \leq i < i_{k+1}} \hat{p}_{\hat{\sigma}(i)}(c_k(p_k, i) \mid x; \theta),$$

where $k(i)$ denotes the segment containing event i . Summing over the joint hypothesis space $(p_1, \dots, p_\kappa) \in \{0, \dots, L_1 - 1\} \times \dots \times \{0, \dots, L_\kappa - 1\}$ of a product with this structure is exactly a sum of a product of independent factors, which itself factors into a product of independent sums,

$$\sum_{p_1, \dots, p_\kappa} \prod_{k=1}^{\kappa} f_k(p_k) = \prod_{k=1}^{\kappa} \left(\sum_{p_k} f_k(p_k) \right),$$

applied here with $f_k(p_k) := \prod_{i: i_k \leq i < i_{k+1}} \hat{p}_{\hat{\sigma}(i)}(c_k(p_k, i) \mid x; \theta)$. Taking the logarithm converts this product of κ independent sums into a sum of κ independent log-sums:

$$\log \sum_{p_1, \dots, p_\kappa} \prod_{i=1}^M \hat{p}_{\hat{\sigma}(i)}(c(p_{k(i)}, i) \mid x; \theta) = \sum_{k=1}^{\kappa} \log \sum_{p=0}^{L_k-1} \prod_{i: i_k \leq i < i_{k+1}} \hat{p}_{\hat{\sigma}(i)}(c_k(p, i) \mid x; \theta). \quad (16)$$

This is exactly κ separate copies of eq. (11)’s own last term, one per segment, each using only that segment’s own matched events and its own meter L_k — not an approximation to the single-meter derivation, but the same derivation applied once per segment and summed, licensed entirely by the factoring already visible before any sum was taken.

The E-step, M-step, and training loss are each unchanged in form. Because eq. (16) is a sum of κ independent terms, the lower bound $\mathcal{F}(q, \theta)$, its Jensen-tight choice of q , and the resulting $Q(\theta; \theta_{\text{old}})$ all decompose additively across segments by the identical argument used for the single-segment case, applied independently within each summand of eq. (16). Concretely, segment k gets its own E-step posterior,

$$\pi_p^{(k)} := P(\phi_{i_k} = p \mid y_{1:M}, x; \theta_{\text{old}}) = \frac{\prod_{i: i_k \leq i < i_{k+1}} \hat{p}_{\hat{\sigma}(i)}(c_k(p, i) \mid x; \theta_{\text{old}})}{\sum_{p'=0}^{L_k-1} \prod_{i: i_k \leq i < i_{k+1}} \hat{p}_{\hat{\sigma}(i)}(c_k(p', i) \mid x; \theta_{\text{old}})}, \quad p = 0, \dots, L_k-1, \quad (17)$$

computed entirely from segment k ’s own matched events, with no reference to any other segment’s predictions. Each event i in segment k then gets exactly the same marginal downbeat probability as before, $r_i(\theta_{\text{old}}) := \sum_{p: c_k(p, i) = \text{DB}} \pi_p^{(k)}$, and the resulting M-step objective is exactly eq. (15), summed over all M matched events regardless of which segment each belongs to — *the training-loop’s own per-event loss formula never changes*. What changes under mixed meter is entirely upstream of it: how $r_i(\theta_{\text{old}})$ is computed, namely from (17) restricted to event i ’s own segment, rather than from a single fragment-wide posterior. This is a genuine practical simplification, not merely a notational one: implementing mixed-meter support requires changing only the E-step computation feeding r_i into Algorithm 3, not the loss itself or the M-step that consumes it.

Cost. Computing every segment’s own $\pi^{(k)}$ costs $O(\sum_k L_k \cdot |\{i : i_k \leq i < i_{k+1}\}|)$, the same $O(L \cdot M)$ -shaped cost as the single-segment case, now distributed across segments rather than incurred once — no increase in asymptotic complexity, since every matched event still participates in exactly one segment’s computation. This mirrors Section 4.2’s own finding for the fully-labeled, matching-affecting construction there: incorporating per-segment meter structure into an already-derived quantity is cheap, precisely because segments do not interact.

Remark 2 (This is not a latent-variable formulation of subset selection). *The latent variable Z_i implicit above (whether event i is DB or B) operates strictly downstream of, and conditional on, an already-fixed $\hat{\sigma}$ from Section 5: it concerns only the class label of a pair that the hard dynamic program has already matched, not which pairing was chosen. σ itself remains, throughout this subsection, exactly what Section 5’s Proposition and Section 7 establish it to be — the exact mode of the posterior $P(\sigma \mid y, \theta, x)$ derivable from Section 4’s observation model under a uniform prior, held fixed and detached from the gradient while it serves that role. These are latent variables at two different levels of the same pipeline, answering two different questions: Z_i asks which of two known labels a given matched pair has; σ asks which candidates are matched at all. Conflating the two would misstate what Section 5’s DP computes. Section 8.4 below constructs a model in which the full posterior over σ , not only its mode, is used; it is a different, more invasive construction than the one in this subsection, not a restatement of it.*

8.2 Decomposing the E-Step: Matching First, Phase Second

It is worth stating explicitly what Sections 7–8 have, in effect, already done, since the two sections’ constructions only make sense together as a single decomposition, not as two independent devices. The fragment’s full latent structure is the pair (σ, ϕ_0) — which candidates are matched, and what phase the matched events’ downbeat pattern starts at (writing ϕ_0 for the single-meter case; Section 8.1’s $(\phi_{i_1}, \dots, \phi_{i_\kappa})$ generalizes this to a mixed-meter fragment, with everything below applying identically to each segment’s own phase-origin). Rather than infer (σ, ϕ_0) jointly, this document’s E-step is split into two substeps, run in sequence: a first substep that infers $\hat{\sigma}$ alone, by the hard dynamic program of Section 5, using a cost (3) built only from each candidate’s own class and time prediction, with no reference to ϕ_0 anywhere in it; and a second substep, Section 8’s own E-step (12), that infers the posterior over ϕ_0 *given* $\hat{\sigma}$ already fixed, using only the matched candidates $\hat{\sigma}(1), \dots, \hat{\sigma}(M)$ that the first substep selected.

This is a genuine answer to the question of whether (σ, ϕ_0) can be decomposed this way: yes, and this document already does so throughout, not merely in principle. It is worth being equally explicit, though, about what the decomposition costs, since a sequential E-step is not a free simplification of a joint one.

The first substep cannot use phase evidence to disambiguate matching. $\mathcal{L}_{\text{match}}(y_i, \hat{y}_j)$ (3) is built entirely from $\hat{p}_j(c_i \mid \cdot)$ and $|t_i - \hat{t}_j(\theta, x)|$ — quantities available before any phase hypothesis is ever considered. Concretely, this means: if two neighboring candidates are roughly equally plausible matches for event i by class and timing alone, but only one of them would sit at a metrically sensible phase given the pattern the *other* matched events already establish, the first substep has no way to prefer it — phase evidence is computed only in the second substep, from candidates the first substep has already committed to.

The first substep’s decision cannot be revised by the second. Once $\hat{\sigma}$ is fixed (Algorithm 1, line 16, and the stop-gradient of Algorithm 3), the second substep’s own posterior (12) is computed entirely from the matched candidates $\hat{\sigma}(1), \dots, \hat{\sigma}(M)$ that the first substep already chose; nothing in eq. (12) or the M-step that follows it can feed evidence back to reconsider whether some *different* candidate should have been matched to event i in the first place, even if the resulting phase pattern would reveal the original match as metrically implausible. The two substeps communicate in one direction only.

Why Section 8.4 alone does not resolve this. Section 8.4 constructs an alternative in which σ ’s own posterior, rather than its mode, participates in training — a step toward joint treatment of the two latents, though it too computes ϕ_0 ’s own posterior only after σ ’s distribution (not a fixed value, but still a marginal over σ alone) has already been formed, and does not couple the two inference problems within a single recursion. Section 8.3 below takes this on directly for the hard-EM case, and shows the resulting construction is considerably cheaper than it might appear.

8.3 A Genuinely Joint Solution: Coupling Matching and Phase in One Computation

The two limitations named above both trace to the same cause: $\mathcal{L}_{\text{match}}(y_i, \hat{y}_j)$ (3) is evaluated before any phase hypothesis exists, so it cannot use one. The fix is to make the per-pair cost itself depend on a phase hypothesis, and to search over hypotheses and matchings together rather than one after the other.

A phase-conditioned cost. For a beat-only event i and a hypothesized $\phi_0 = p$, define

$$\mathcal{L}_{\text{match}}^p(y_i, \hat{y}_j) := -\log \hat{p}_j(\phi_j = (p + i - 1) \bmod L \mid x; \theta) + \lambda_{L1} |t_i - \hat{t}_j(\theta, x)|, \quad (18)$$

using the phase-valued classification probability of Section 8, now applied to a generic candidate j rather than an already-fixed $\hat{\sigma}(i)$, since matching itself is what this cost feeds into and is not yet determined — $\phi_j = (p + i - 1) \bmod L$ is the phase candidate j would carry were it matched to event i under hypothesis p . This is not a new construction so much as eq. (3) generalized: for a fully-labeled event, where c_i is directly observed rather than hypothesized, $\mathcal{L}_{\text{match}}^p$ plays no role at all and eq. (3) is used unchanged.

The key structural fact: phase propagation is deterministic, not a genuine branching choice. Section 8’s own opening already established why: since the phase increment counts matched events, never candidates, the entire sequence $\phi_1, \dots, \phi_M$ is fixed the moment ϕ_0 is, $\phi_i = (\phi_0 + i - 1) \bmod L$ for every i . There is no additional latent choice anywhere past ϕ_0 , and no restart or branching of the kind a genuinely Markov-transitioning phase chain would need. This is what makes the construction below considerably simpler than it might first appear.

Joint MAP reduces to L independent reruns of Algorithm 1. Fix a hypothesis $\phi_0 = p$. Every candidate’s hypothesized phase $(p + i - 1) \bmod L$ is then already determined for each event i , so $\mathcal{L}_{\text{match}}^p$ is an ordinary per-pair cost of exactly the shape Section 5’s dynamic program minimizes. Running Algorithm 1 with $\mathcal{L}_{\text{match}}^p$ in place of $\mathcal{L}_{\text{match}}$ gives $\hat{\sigma}^{(p)}$ and its optimal cost $D^{(p)}[M, N]$ — the best matching *given* that phase starts at p . The joint MAP estimate of (σ, ϕ_0) together is then

$$\hat{\phi}_0 := \arg \min_{p \in \{0, \dots, L-1\}} D^{(p)}[M, N], \quad \hat{\sigma} := \hat{\sigma}^{(\hat{\phi}_0)}. \quad (19)$$

That is: run the existing dynamic program L times, once per phase hypothesis, and keep whichever run achieved the lowest total cost, together with the matching it produced. We verified this construction directly against brute-force enumeration of every (σ, ϕ_0) pair on a minimal case ($M = 2, N = 3, L = 2$): the two methods agree exactly, and — confirming the construction is genuinely coupling the two latents rather than computing them independently and combining the results after the fact — the optimal $\hat{\sigma}$ differed between the two phase hypotheses in that example, since a candidate whose own class prediction favors DB is a cheap match for an event one hypothesis calls a downbeat, and an expensive match for the same event under a hypothesis that calls it a beat.

This resolves both limitations named above. A candidate’s own phase-consistency with the pattern under consideration now directly affects whether it is the cheapest match for a given event, resolving the first limitation; and because $\hat{\sigma}$ is re-derived for every phase hypothesis rather than fixed once and reused, a matching that looks attractive under one hypothesis but metrically implausible under the hypothesis that turns out to win is never the one reported — resolving the second. Both were, in Section 8.2’s own terms, consequences of computing $\hat{\sigma}$ before any phase evidence existed to inform it; eq. (19) does not do that.

Correcting the earlier scope claim. The paragraph opening this section previously stated that a construction inferring (σ, ϕ_0) jointly would be a substantially larger undertaking than either device already presented. For the single, known-meter case, this was an overstatement, corrected here rather than left standing: eq. (19) costs $O(L \cdot NM)$, exactly L ordinary runs of Algorithm 1, a modest constant-factor increase over the sequential approach’s $O(NM)$ (Section 5) plus $O(LM)$

(Section 8’s own E-step) — not a fundamentally more expensive kind of computation, and well within reach for the small L values (typically 2 to 12) beat tracking actually uses.

Extending to mixed meter. Section 8.1’s segments do not admit quite this simple a reduction, because σ ’s own order-preservation is a global constraint: which candidates remain available to segment $k + 1$ depends on where segment k ’s own matching ended, so the κ segments cannot each be solved by an independent rerun without losing track of that boundary. The fix generalizes Section 4.2’s own augmented-state device — there used to carry a downbeat’s matched candidate identity across a spacing constraint, here used to carry the current phase across a segment boundary — rather than introducing a new technique. Define, for ϕ ranging over $\{0, \dots, L_{k(i)} - 1\}$ where $k(i)$ is the segment containing event i ,

$D[i, j, \phi] :=$ minimum cost of an order-preserving injection from $\{1, \dots, i\}$ into $\{1, \dots, j\}$ with event i ’s own phase with recursion

$$D[i, j, \phi] = \min \left(D[i, j - 1, \phi], D[i - 1, j - 1, \phi'] - \log \hat{p}_{j-1}(\phi_{j-1} = \phi \mid x; \theta) + \lambda_{L1} |t_i - \hat{t}_{j-1}(\theta, x)| \right), \quad (20)$$

where, if i is not the first event of its segment, ϕ' is the unique value with $(\phi' + 1) \bmod L_{k(i)} = \phi$ (phase propagation within a segment remains deterministic, so there is genuinely only one such ϕ' to consider, not a minimum over several); and if $i = i_k$ is the first event of a new segment, the match branch instead uses $\min_{\phi'} D[i - 1, j - 1, \phi']$, since segment k ’s own phase-origin hypothesis ϕ is chosen independently of whatever phase segment $k - 1$ happened to end on. This costs $O(NM \cdot L_{\max})$ for $L_{\max} := \max_k L_k$ — the same modest, constant-factor increase as the single-meter case above, generalized rather than made asymptotically worse.

The sequential and joint constructions, compared directly. Section 8.2 confirmed the sequential decomposition is a genuine answer to whether (σ, ϕ_0) can be inferred one after the other: yes, and this document does so throughout Sections 7–8. What that section left as two separately-stated costs is worth drawing together here, now that the joint alternative has been built and both are available for direct comparison.

The two constructions solve different objectives, not the same objective by different means. Sequential: $\hat{\sigma} := \arg \min_{\sigma} \sum_i \mathcal{L}_{\text{match}}(y_i, \hat{y}_{\sigma(i)})$ using the phase-blind cost (3), followed by π_p computed from that fixed $\hat{\sigma}$ alone (12). Joint: $(\hat{\sigma}, \hat{\phi}_0) := \arg \min_{\sigma, p} \sum_i \mathcal{L}_{\text{match}}^p(y_i, \hat{y}_{\sigma(i)})$ (19), matching and phase resolved together against the phase-conditioned cost (18) directly. The sequential $\hat{\sigma}$ is therefore not an approximation to the joint $\hat{\sigma}$ computed more cheaply; it is the exact minimizer of a genuinely different cost, one with no phase term to minimize over in the first place, since $\mathcal{L}_{\text{match}}$ cannot reference ϕ_0 by construction, not by omission.

This difference in objective is what produces Section 8.2’s two named failure modes, not something separate from them. A candidate that is a middling match by class and timing alone but would sit at a metrically implausible phase given the rest of the pattern has no way to be penalized for that implausibility under $\mathcal{L}_{\text{match}}$, since phase does not exist yet when that cost is evaluated — the joint cost $\mathcal{L}_{\text{match}}^p$ penalizes it directly, being evaluated only once a phase hypothesis is already in hand. And because the sequential decomposition commits to $\hat{\sigma}$ before phase is ever considered, no later phase evidence can revise that commitment, even when the resulting phase pattern reveals the original match as implausible after the fact — the joint construction cannot exhibit this failure mode, since $\hat{\sigma}$ and $\hat{\phi}_0$ are chosen together, with neither fixed in advance of the other.

The price is a bounded, not asymptotic, increase in cost: $O(NM)$ for the sequential construction against $O(L \cdot NM)$ for the joint one, the L independent reruns of Algorithm 1 established above. This is exact, not approximate, coupling purchased at a modest constant-factor cost — confirmed directly in Section 8.3’s own numerical check, where the optimal $\hat{\sigma}$ genuinely differed between phase hypotheses on a minimal worked example, ruling out the possibility that this coupling is a correction that rarely matters in practice.

What remains open. This construction is a hard-EM treatment of ϕ_0 jointly with σ — it produces a single joint MAP estimate $(\hat{\sigma}, \hat{\phi}_0)$, not a posterior over either. Section 8.5 below completes the remaining piece: combining this with Section 8.4’s own soft treatment of σ , so that both σ and ϕ_0 are marginalized together rather than either being hardened.

8.4 A Model with Genuinely Latent Subset Selection, Trained by Marginal Likelihood

The construction below replaces the hard subset-selection step of Sections 4–7 with a construction that retains the full posterior distribution over σ , rather than only its mode, used to define the training loss itself — not a device applied only after training to interpret or post-process a fixed $\hat{\theta}$. Training under this section’s loss means differentiating a marginal likelihood that sums over every order-preserving injection, rather than differentiating a loss evaluated at one hard-selected σ with the selection step detached from the gradient. Everything about the feasible set is unchanged from Section 4: σ still ranges only over $\mathfrak{D}_{M,N}$, and the monotonicity guarantee of Section 3 is untouched, since nothing here modifies how $\hat{t}_j$ is computed.

A correction the marginalized version makes unavoidable. Write $g_j(\theta, x) := -\log \hat{p}_j(\emptyset \mid x; \theta)$ for the cost of candidate j predicting background. The full training loss of Section 7, unlike the per-pair cost (3) that Section 5’s dynamic program minimizes, includes the background term $\gamma \sum_{j \notin \text{im}(\sigma)} g_j(\theta, x)$. Because $|\text{im}(\sigma)| = M$ is fixed for every $\sigma \in \mathfrak{D}_{M,N}$, this term is not independent of σ : $\sum_{j \notin \text{im}(\sigma)} g_j(\theta, x) = \sum_{j=0}^{N-1} g_j(\theta, x) - \sum_{i=1}^M g_{\sigma(i)}(\theta, x)$, so minimizing the full loss over σ is equivalent to minimizing

$$\sum_{i=1}^M \left[\mathcal{L}_{\text{match}}(y_i, \hat{y}_{\sigma(i)}) - \gamma g_{\sigma(i)}(\theta, x) \right],$$

not $\sum_i \mathcal{L}_{\text{match}}(y_i, \hat{y}_{\sigma(i)})$ alone as written in equation (4). The hard dynamic program of Section 5, as stated, therefore does not exactly minimize the full training loss of Section 7 once the background term is accounted for; it minimizes only the matched-pair portion of it. This is a genuine gap in the hard formulation as presented, not merely a simplification introduced here, and it is corrected by the same drop-in substitution in both the hard and the marginalized version: replace $\mathcal{L}_{\text{match}}(y_i, \hat{y}_j)$ with the corrected cost $\mathcal{L}'_{\text{match}}(y_i, \hat{y}_j) := \mathcal{L}_{\text{match}}(y_i, \hat{y}_j) - \gamma g_j(\theta, x)$ everywhere it is used to compare candidate assignments. We use $\mathcal{L}'_{\text{match}}$ throughout the remainder of this subsection.

A distribution over order-preserving injections, and its partition function. Define a Gibbs distribution over $\mathfrak{D}_{M,N}$ using $\mathcal{L}'_{\text{match}}$ as an energy,

$$P(\sigma \mid x; \theta) = \frac{1}{Z(\theta, x)} \exp \left(- \sum_{i=1}^M \mathcal{L}'_{\text{match}}(y_i, \hat{y}_{\sigma(i)}) \right), \quad Z(\theta, x) = \sum_{\sigma' \in \mathfrak{D}_{M,N}} \exp \left(- \sum_{i=1}^M \mathcal{L}'_{\text{match}}(y_i, \hat{y}_{\sigma'(i)}) \right), \quad (21)$$

the sum in $Z(\theta, x)$ ranging over every one of the $\binom{N}{M}$ order-preserving injections $\sigma' \in \mathfrak{D}_{M,N}$. Since $\mathcal{L}'_{\text{match}}(y_i, \hat{y}_j) = \mathcal{L}_{\text{match}}(y_i, \hat{y}_j) - \gamma g_j(\theta, x)$, expanding gives $\exp(-\mathcal{L}'_{\text{match}}(y_i, \hat{y}_j)) = p(y_i | \hat{y}_j, \theta) \cdot \hat{p}_j(\emptyset | x; \theta)^{-\gamma}$: the observation-model probability from (2), multiplied by an additional factor depending on candidate j 's own predicted background probability. $Z(\theta, x)$ is therefore not the marginal likelihood of observation model (2) under a uniform prior, but the marginal likelihood of a related model in which the effective prior over σ is this reweighted quantity rather than uniform. It remains the discrete analogue of marginalizing over the unknown correspondence — the same latent function σ introduced in Section 4, treated here in full rather than reduced to its mode — rather than committing to its single most likely value, but for this reweighted model, not model (2) as stated.

$Z(\theta, x)$ is computable exactly, in the same asymptotic complexity as Section 5's dynamic program, by replacing the minimum in the recursion (7) with a log-sum-exp:

$$\tilde{D}[i, j] = \text{logsumexp} \left(\tilde{D}[i, j-1], \tilde{D}[i-1, j-1] - \mathcal{L}'_{\text{match}}(y_i, \hat{y}_{j-1}) \right), \quad \tilde{D}[0, j] = 0, \quad \tilde{D}[i, 0] = -\infty \quad (i \geq 1), \quad (22)$$

with $\log Z(\theta, x) = \tilde{D}[M, N]$. This is the exact structured-prediction analogue of the relationship between Viterbi decoding and the forward algorithm for hidden Markov models: Section 5's recursion computes the minimum-cost (most probable) σ under the tropical semiring, while (22) computes the total probability mass over all of $\mathfrak{D}_{M,N}$ under the ordinary sum-product semiring, using the identical recursive structure and the identical $O(NM)$ complexity.

Training loss and procedure. The training loss for a fragment is the negative log marginal likelihood,

$$\mathcal{L}_{\text{marg}}(\theta) = -\log Z(\theta, x) = -\tilde{D}[M, N], \quad (23)$$

dropping the additive, θ -independent constant from the choice of prior over $\mathfrak{D}_{M,N}$. Unlike Section 7's loss, (23) requires no stop-gradient: every quantity in (22) is differentiable, and $\log Z(\theta, x)$ is backpropagated through directly, exactly as the partition function of a linear-chain conditional random field is backpropagated through its own forward algorithm during training (Lafferty et al., 2001). This is the central sense in which this construction is a training-time procedure: θ is updated by descending the gradient of (23) itself, with the correspondence between predictions and ground truth marginalized out rather than hardened into a detached $\hat{\sigma}$ before any gradient is taken.

An equivalent EM perspective exists alongside direct backpropagation, exactly paralleling Section 8's own E-step/M-step derivation: the posterior $P(\sigma | y_{1:M}, x; \theta)$ is a genuine distribution obtainable from a forward-backward pass over the same recursion, and by Fisher's identity a single gradient step on the resulting EM surrogate, with the posterior frozen at θ^{old} , coincides exactly with a single gradient step on (23) directly. The two are optimization strategies for the same objective, not competing models; direct backpropagation through (22) is the simpler of the two to implement and is what we recommend. Section 7 and this section are consequently two E-steps for one latent-function model, not two separate models of σ : Section 7 collapses $P(\sigma | y, \theta, x)$ to a point mass (hard-EM, equivalently Dirac EM), and this section retains it in full — the underlying latent object is the same σ introduced in Section 4 in both cases.

Inference still requires a hard decision. Marginalizing σ changes what is optimized during training; it does not change the fact that reporting beat and downbeat locations requires a discrete list of times. At inference, decoding reverts to Section 5's hard dynamic program (Algorithm 1) evaluated at the trained $\hat{\theta}$, exactly as Algorithm 5 already does, or alternatively to thresholding the per-candidate marginal probability of being matched, obtainable from the same forward-backward pass used for the EM perspective above. This mirrors the standard relationship between training

and decoding in structured models trained by marginal likelihood — a hidden Markov model trained by Baum–Welch is still decoded by Viterbi at test time — and should be read as a statement about how this section’s construction is used, not an oversight: the latent-variable treatment of σ is a training-time device.

Confirmation-bias risk and mitigations. Bootstrapping labels from the model’s own predictions risks reinforcing early mistakes, particularly before the shared head has learned a reliable B/DB distinction from the labeled portion of the training data. Two mitigations, both already used for the same underlying reason elsewhere in this document: a warm-up phase training exclusively on fully-labeled data (or on (9) alone) before introducing (15), mirroring the precision-parameter warm-up of Section 4.1; and confidence gating, using (15) only when $\max(r_i(\theta_{\text{old}}), 1 - r_i(\theta_{\text{old}}))$ exceeds a fixed threshold and falling back to (9) otherwise, so that a genuinely uncertain instance is not force-fit to an unreliable pseudo-label.

8.5 A Fully Joint Soft Model: Marginalizing (σ, ϕ_0) Together

Section 8.3 found the joint MAP of (σ, ϕ_0) but reduced σ to a point mass in doing so; Section 8.4 kept σ ’s full posterior but never let ϕ_0 participate in it. Neither, on its own, is matching-and-phase-together in the strongest sense available: a single object marginalizing over both simultaneously. This section constructs that object directly, by combining the two devices already given rather than introducing new machinery.

A phase-and-background-corrected cost. Recall $\mathcal{L}_{\text{match}}^p(y_i, \hat{y}_j)$ from eq. (18), the phase-conditioned generalization of $\mathcal{L}_{\text{match}}$, and $g_j(\theta, x) := -\log \hat{p}_j(\emptyset \mid x; \theta)$, the background cost from Section 8.4. Define

$$\mathcal{L}_{\text{match}}^{'p}(y_i, \hat{y}_j) := \mathcal{L}_{\text{match}}^p(y_i, \hat{y}_j) - \gamma g_j(\theta, x) = -\log \hat{p}_j(\phi_j = (p+i-1) \bmod L \mid x; \theta) + \lambda_{L1} |t_i - \hat{t}_j(\theta, x)| - \gamma g_j(\theta, x),$$

combining both corrections already established separately: phase-conditioning from Section 8.3, since σ is no longer fixed before phase is considered; and the background correction from Section 8.4, needed for exactly the reason derived there, unaffected by whether σ or ϕ_0 is also being marginalized.

A joint Gibbs distribution over σ and ϕ_0 together. Define

$$P(\sigma, \phi_0 = p \mid x; \theta) = \frac{1}{Z_{\text{joint}}(\theta, x)} \exp \left(-\sum_{i=1}^M \mathcal{L}_{\text{match}}^{'p}(y_i, \hat{y}_{\sigma(i)}) \right), \quad Z_{\text{joint}}(\theta, x) = \sum_{p=0}^{L-1} \sum_{\sigma \in \mathfrak{D}_{M,N}} \exp \left(-\sum_{i=1}^M \mathcal{L}_{\text{match}}^{'p}(y_i, \hat{y}_{\sigma(i)}) \right) \quad (24)$$

a single distribution over the combined space $\mathfrak{D}_{M,N} \times \{0, \dots, L-1\}$, rather than a distribution over σ alone or a point estimate of ϕ_0 alone.

The joint partition function decomposes into L independent copies of Section 8.4’s own construction. For a fixed p , the inner sum over $\sigma \in \mathfrak{D}_{M,N}$ in eq. (24) is exactly $Z(\theta, x)$ from eq. (21), with $\mathcal{L}_{\text{match}}^{'p}$ in place of $\mathcal{L}'_{\text{match}}$; write $Z^{(p)}(\theta, x)$ for this quantity, computable exactly by Section 8.4’s own log-sum-exp recursion (22) with the substitution made. Then

$$Z_{\text{joint}}(\theta, x) = \sum_{p=0}^{L-1} Z^{(p)}(\theta, x), \quad (25)$$

the direct soft analogue of eq. (19): there, the joint MAP was the *minimum* of L independent hard costs; here, the joint marginal likelihood is the *sum* of L independent soft partition functions — the same relationship as between Viterbi decoding and the forward algorithm that Section 8.4 already invoked, now applied one level up, over ϕ_0 as well as σ .

Concrete posterior quantities. Eq. (24) defines a distribution, but stops short of the quantities actually usable downstream — the analogues of $r_i(\theta_{\text{old}})$ from Section 8, now for the fully joint case. We derive three, each checked directly against brute-force enumeration on a minimal case before being stated here.

The posterior weight of a single phase hypothesis follows directly from the decomposition (25):

$$P(\phi_0 = p \mid x; \theta) = \frac{Z^{(p)}(\theta, x)}{Z_{\text{joint}}(\theta, x)}. \quad (26)$$

Because $\phi_{\hat{\sigma}(i)}$'s hypothesized value $(p+i-1) \bmod L$ depends only on (p, i) , never on which candidate σ happens to assign to event i , event i 's own downbeat marginal requires no forward-backward pass at all — and since $\{p : (p+i-1) \bmod L = 0\} = \{p_i^*\}$ is a singleton (the same $p_i^* = (1-i) \bmod L$ from Section 8), it is a direct lookup of eq. (26), not a sum:

$$r_i(\theta) := P(C_i = \text{DB} \mid x; \theta) = P(\phi_0 = p_i^* \mid x; \theta), \quad (27)$$

the direct generalization of eq. (14) to a setting where σ is no longer fixed: ϕ_0 's own posterior already fully determines r_i , exactly as it did when σ was held fixed, because σ was never part of what determines a hypothesis's own downbeat pattern in the first place.

The matching marginal $P(\sigma(i) = j \mid x; \theta)$ does require coupling to σ , and hence a genuine forward-backward pass within each phase hypothesis. Define the backward companion to eq. (22), for a fixed p (writing $\mathcal{L}'^p_{\text{match}}$ for whichever cost is in force, single-meter or otherwise):

$$\tilde{D}_p^{\text{bwd}}[i, j] = \text{logsumexp} \left(\tilde{D}_p^{\text{bwd}}[i, j+1], \tilde{D}_p^{\text{bwd}}[i+1, j+1] - \mathcal{L}'^p_{\text{match}}(y_{i+1}, \hat{y}_j) \right), \quad \tilde{D}_p^{\text{bwd}}[M, j] := 0, \quad \tilde{D}_p^{\text{bwd}}[i, N] := -\infty \quad (28)$$

summing, in log space, over every way to complete a valid injection from $\{i+1, \dots, M\}$ into candidates $\{j, \dots, N-1\}$ — the match branch uses candidate j directly, not $j+1$, since j marks where the remaining, not-yet-considered candidates begin under the 0-indexed labeling. Combining the forward table $\tilde{D}_p[i, j]$ (22) with $\tilde{D}_p^{\text{bwd}}[i, j]$ gives the matching marginal within one fixed hypothesis,

$$P(\sigma(i) = j \mid \phi_0 = p, x; \theta) = \exp \left(\tilde{D}_p[i-1, j] - \mathcal{L}'^p_{\text{match}}(y_i, \hat{y}_j) + \tilde{D}_p^{\text{bwd}}[i, j+1] - \tilde{D}_p[M, N] \right),$$

and mixing over the phase posterior (26) gives the fully marginal quantity,

$$P(\sigma(i) = j \mid x; \theta) = \sum_{p=0}^{L-1} P(\phi_0 = p \mid x; \theta) \cdot P(\sigma(i) = j \mid \phi_0 = p, x; \theta). \quad (29)$$

We verified eqs. (26)–(29) directly against brute-force enumeration of the full joint distribution on the same minimal case as before: all three match exactly, including the closed-form r_i requiring no forward-backward and the forward-backward-derived matching marginal.

Training loss. The fragment’s contribution to the training loss is

$$\mathcal{L}_{\text{joint-marg}}(\theta) := -\log Z_{\text{joint}}(\theta, x) = -\log \sum_{p=0}^{L-1} Z^{(p)}(\theta, x), \quad (30)$$

requiring no stop-gradient anywhere: every quantity entering eq. (30), from the L calls to the log-sum-exp recursion through the final sum and logarithm, is differentiable, and θ is updated by descending $\nabla_{\theta} \mathcal{L}_{\text{joint-marg}}(\theta)$ directly. This is the fully joint counterpart to Section 8.4’s own $\mathcal{L}_{\text{marg}}(\theta)$ (23), now marginalizing ϕ_0 alongside σ rather than leaving it out of that marginalization entirely.

Verification. We checked eq. (25) directly against brute-force enumeration of every (σ, ϕ_0) pair on a minimal case ($M = 2, N = 3, L = 2$, with the background correction $\gamma = 0.5$ included): summing $\exp(-\sum_i \mathcal{L}'^p_{\text{match}}(y_i, \hat{y}_{\sigma(i)}))$ over every one of the $L \cdot \binom{N}{M}$ pairs directly matches $\sum_p Z^{(p)}(\theta, x)$ computed via eq. (25) to full numerical precision; and each individual $Z^{(p)}(\theta, x)$ matches its own log-sum-exp dynamic-programming computation exactly, confirming both the decomposition and the recursion it relies on.

Inference and the mixed-meter extension. As in Section 8.4, marginalizing (σ, ϕ_0) changes what training optimizes, not what decoding must eventually produce: at inference there is no y , hence no $\mathcal{L}'^p_{\text{match}}$ to form and no posterior over (σ, ϕ_0) to compute, and decoding proceeds exactly as Section 9.2 already specifies. The mixed-meter generalization of Section 8.1 carries over the same way it did for the hard case in Section 8.3: eq. (16)’s own factorization across segments means $Z_{\text{joint}}(\theta, x)$ becomes a *product* of κ independent per-segment sums, $\prod_{k=1}^{\kappa} \sum_{p=0}^{L_k-1} Z_k^{(p)}(\theta, x)$, each $Z_k^{(p)}$ computed by the augmented log-sum-exp recursion restricted to segment k ’s own matched-candidate range — the soft analogue of eq. (20), at the same $O(NM \cdot L_{\text{max}})$ cost as its hard counterpart.

8.6 Extending to Unknown Meter: Marginalizing L as Well

Everything above, including Section 8.5’s own joint treatment of (σ, ϕ_0) , still assumes L is known, external metadata — the same assumption Section 8.1 states explicitly and relies on throughout. This assumption genuinely fails when a track’s meter is not reliably annotated at all, which is a real, not merely hypothetical, gap: $\phi_{\hat{\sigma}(i)}$ ’s own hypothesized value, $(p + i - 1) \bmod L$, requires L to be fixed before p ’s range is even well-defined, so nothing above is well-formed once L itself is unknown. L belongs in the same category as σ and ϕ_0 — a genuine unobserved quantity requiring inference, not a third exception to the treatment given to the other two.

A finite candidate meter set. Marginalizing over L requires a finite range to sum over, exactly as marginalizing over ϕ_0 required a finite $\{0, \dots, L - 1\}$: fix $\mathcal{M} := \{2, 3, \dots, L_{\text{max}}\}$, covering the simple and compound meters in common use. The phase-valued classification probability of Section 8 already generalizes to this setting without any new notation: for a candidate j and a hypothesized (L, p) pair, the relevant query is simply $\hat{p}_j(\phi_j = (p + i - 1) \bmod L \mid x; \theta)$, with L appearing explicitly inside the modulus rather than needing a separate function of three arguments to carry it.

The doubly-corrected cost, and the doubly-marginalized Gibbs distribution. Define

$$\mathcal{L}'_{\text{match}}{}^{L,p}(y_i, \hat{y}_j) := -\log \hat{p}_j(\phi_j = (p + i - 1) \bmod L \mid x; \theta) + \lambda_{L1} |t_i - \hat{t}_j(\theta, x)| - \gamma g_j(\theta, x),$$

exactly $\mathcal{L}'_{\text{match}}{}^p$ from Section 8.5 with L now written explicitly rather than held fixed, and extend the joint Gibbs distribution over the full space $\mathfrak{D}_{M,N} \times \{0, \dots, L_{\max} - 1\} \times \mathcal{M}$:

$$P(\sigma, \phi_0 = p, L \mid x; \theta) = \frac{1}{Z_{\text{joint}}(\theta, x)} \exp\left(-\sum_{i=1}^M \mathcal{L}'_{\text{match}}{}^{L,p}(y_i, \hat{y}_{\sigma(i)})\right), \quad Z_{\text{joint}}(\theta, x) = \sum_{L \in \mathcal{M}} \sum_{p=0}^{L-1} \sum_{\sigma \in \mathfrak{D}_{M,N}} \exp\left(-\sum_{i=1}^M \mathcal{L}'_{\text{match}}{}^{L,p}(y_i, \hat{y}_{\sigma(i)})\right) \quad (31)$$

By the identical argument as eq. (25), the inner sum over σ for fixed (L, p) is computed exactly by Section 8.4's own log-sum-exp recursion with $\mathcal{L}'_{\text{match}}{}^{L,p}$ substituted; write $Z^{(L,p)}(\theta, x)$ for the result. Then

$$Z_{\text{joint}}(\theta, x) = \sum_{L \in \mathcal{M}} \sum_{p=0}^{L-1} Z^{(L,p)}(\theta, x), \quad (32)$$

a double sum over every (L, p) pair rather than eq. (25)'s single sum over p at one fixed L — the same construction, one level further marginalized. We verified eq. (32) directly against brute-force enumeration over every (L, p, σ) triple on a minimal case ($M = 2, N = 3, \mathcal{M} = \{2, 3\}$): exact agreement, confirming the extension holds without qualification.

Concrete posterior quantities, generalized. Section 8.5's three posterior quantities generalize by summing over $L \in \mathcal{M}$ as well as p , in each case verified against brute-force enumeration on the same two-meter case before being stated here. The joint hypothesis posterior,

$$P(L, \phi_0 = p \mid x; \theta) = \frac{Z^{(L,p)}(\theta, x)}{Z_{\text{joint}}(\theta, x)},$$

gives the meter posterior directly by summing out p ,

$$P(L \mid x; \theta) = \sum_{p=0}^{L-1} P(L, \phi_0 = p \mid x; \theta), \quad (33)$$

and gives $r_i(\theta)$ exactly as before, since $\phi_{\hat{\sigma}(i)}$'s hypothesized value $(p + i - 1) \bmod L$ still depends only on the hypothesis, never on σ , and for each fixed L the inner sum is again a singleton lookup at $p_i^*(L) := (1 - i) \bmod L$ — now depending on L , since which phase makes event i a downbeat shifts with the hypothesized meter — leaving only the outer sum over $L \in \mathcal{M}$ genuine:

$$r_i(\theta) = \sum_{L \in \mathcal{M}} P(L, \phi_0 = p_i^*(L) \mid x; \theta). \quad (34)$$

The matching marginal generalizes the same way: run the backward recursion (28) once per (L, p) pair using $\mathcal{L}'_{\text{match}}{}^{L,p}$, obtain $P(\sigma(i) = j \mid L, \phi_0 = p, x; \theta)$ exactly as in eq. (29)'s own within-hypothesis formula, and mix over the full joint hypothesis posterior,

$$P(\sigma(i) = j \mid x; \theta) = \sum_{L \in \mathcal{M}} \sum_{p=0}^{L-1} P(L, \phi_0 = p \mid x; \theta) \cdot P(\sigma(i) = j \mid L, \phi_0 = p, x; \theta).$$

All three were confirmed exact against brute-force enumeration over every (L, p, σ) triple on the $\mathcal{M} = \{2, 3\}$ case, alongside eq. (32) itself.

No separate prior over L is introduced. Eq. (31) weights every (L, p, σ) triple by $\exp(-\sum_i \mathcal{L}'_{\text{match}}(L, p, \sigma)_i(\cdot))$ alone — implicitly a uniform prior over $\mathcal{M}$, with all discrimination between candidate meters coming from how well each one’s phase-consistent class predictions fit the observed data through the cost itself. A track with strongly periodic classification evidence will have one L ’s own $\sum_p Z^{(L,p)}$ dominate the sum; a track with weak or ambiguous evidence will not. This is a modeling choice, not a derived necessity: a learned meter prior $\hat{q}(L \mid x; \theta)$, of the kind a dedicated meter-classification head could supply, could be incorporated by weighting each $Z^{(L,p)}$ term by $\hat{q}(L \mid x; \theta)$ before summing, but we do not introduce such a head here, since nothing in Section 3’s architecture currently provides one — this is left as a direct extension, not resolved in this document.

Training loss and cost. The training loss is $\mathcal{L}_{\text{joint-marg}}(\theta) := -\log Z_{\text{joint}}(\theta, x)$, unchanged in form from eq. (30), now with Z_{joint} computed from eq. (32). The cost rises from $O(NM \cdot L_{\text{max}})$ to $O(NM \cdot \sum_{L \in \mathcal{M}} L)$, since a separate log-sum-exp recursion is run for every one of the $\sum_{L \in \mathcal{M}} L$ pairs rather than only L_{max} of them; for $\mathcal{M} = \{2, \dots, 12\}$ this sum is 77, a larger but still entirely tractable constant relative to $O(NM)$ alone. Inference is unaffected for the same reason it was unaffected by marginalizing (σ, ϕ_0) : Algorithm 5 never referenced L , σ , or ϕ_0 in the first place, decoding each candidate independently from $\hat{p}_j$ alone.

9 Training and Inference Procedures

9.1 Training

A single training step for one fragment x with ground truth $y_1, \dots, y_M$ proceeds as follows: the encoder and pooling heads produce the candidate predictions $\{\hat{t}_j, \hat{p}_j(\cdot \mid x; \theta)\}_{j=0}^{N-1}$ of Section 3; the pairwise costs $\mathcal{L}_{\text{match}}(y_i, \hat{y}_j)$ of (3) are formed for all i, j ; the dynamic program of (7) is solved to obtain $\hat{\sigma}$; and the parameters are updated by descending the gradient of the loss (8) evaluated at $\hat{\sigma}$ held fixed. We factor the first of these — computing $\{\hat{t}_j, \hat{p}_j(\cdot \mid x; \theta)\}_{j=0}^{N-1}$ from θ, x alone — into its own named procedure, FORWARD (Algorithm 2 below), since both the E-step and the M-step of Algorithm 3 need its output and neither produces it: making the call explicit in each block, rather than assuming the quantities are simply available, keeps every block’s own dependencies visible in the pseudocode itself. Algorithm 3 performs one full training step for one fragment; Algorithm 4 immediately after it is the outer procedure that calls Algorithm 3 once per fragment, repeated across the full training set and across multiple epochs, and is what is actually run to produce a trained $\hat{\theta}$.

Algorithm 2 FORWARD: candidate predictions from θ, x alone

Require: audio window x ; parameters θ

```
1:  $h \leftarrow \text{Encoder}(x; \theta)$  ▷ frame-level features
2: for  $j = 0, \dots, N - 1$  do
3:    $z_j \leftarrow \text{Downsample}(h)_j$ 
4:    $r_j, \ell_j \leftarrow \text{Heads}(z_j; \theta)$  ▷  $r_j \in \mathbb{R}$  raw output;  $\ell_j$  class logits
5:    $\hat{p}_j(\cdot | x; \theta) \leftarrow \text{softmax}(\ell_j)$ 
6: end for
7:  $Z \leftarrow \sum_{k=0}^{N-1} \text{softplus}(r_k)$  ▷ normalizer, equation (1)
8: for  $j = 0, \dots, N - 1$  do
9:    $\hat{t}_j \leftarrow \frac{1}{Z} \sum_{k=0}^j \text{softplus}(r_k)$  ▷ equation (1)
10: end for
11: return  $\{\hat{p}_j(\cdot | x; \theta), \hat{t}_j\}_{j=0}^{N-1}$ 
```

Algorithm 3 TRAINSTEP: training step (single window)

Require: audio window x ; ground truth $y_1, \dots, y_M$ with $t_1 < \dots < t_M$; parameters θ ; learning rate η ; weights $\lambda_{L1}, \gamma, \{\omega_c\}$

```
1: E-step { ▷ MAP estimate of  $\sigma$  under the current  $\theta$ 
2:    $\{\hat{p}_j(\cdot | x; \theta), \hat{t}_j\}_{j=0}^{N-1} \leftarrow \text{FORWARD}(x, \theta)$  ▷ Algorithm 2
3: for  $i = 1, \dots, M$  do
4:   for  $j = 0, \dots, N - 1$  do
5:      $\mathcal{L}_{\text{match}}(y_i, \hat{y}_j) \leftarrow -\log \hat{p}_j(c_i | x; \theta) + \lambda_{L1} |t_i - \hat{t}_j|$  ▷ equation (3); used only by
     SUBSETSELECTDP below, nowhere else
6:   end for
7: end for
8:  $\hat{\sigma} \leftarrow \text{SUBSETSELECTDP}(\mathcal{L}_{\text{match}})$  ▷ Algorithm 1
9: }
10: M-step { ▷  $\hat{\sigma}$  held fixed while  $\theta$  is updated
11:    $\{\hat{p}_j(\cdot | x; \theta), \hat{t}_j\}_{j=0}^{N-1} \leftarrow \text{FORWARD}(x, \theta)$  ▷ Algorithm 2; same  $\theta$  as the E-step's own call above
12:    $\hat{\sigma} \leftarrow \text{STOPGRADIENT}(\hat{\sigma})$  ▷ no gradient through the selection step
13:    $\mathcal{L}(\theta) \leftarrow \sum_{i=1}^M \left[ \omega_{c_i} \left( -\log \hat{p}_{\hat{\sigma}(i)}(c_i | x; \theta) \right) + \lambda_{L1} |t_i - \hat{t}_{\hat{\sigma}(i)}(\theta, x)| \right] + \gamma \sum_{j \notin \text{im}(\hat{\sigma})} -\log \hat{p}_j(\emptyset | x; \theta)$  ▷
     eq. (8); built from  $\hat{p}_{\hat{\sigma}(i)}, \hat{t}_{\hat{\sigma}(i)}$  directly, not from  $\mathcal{L}_{\text{match}}$ 
14:    $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(\theta)$  ▷ or an Adam-type update
15: }
16: return  $\theta$ 
```

Algorithm 3 brackets the E-step and M-step with their own opening and closing braces so exactly which lines belong to each is visible in the pseudocode itself, not only in this surrounding prose, and each block opens with its own call to FORWARD (Algorithm 2) rather than assuming $\hat{p}_j, \hat{t}_j$ are simply available: every quantity a block needs is obtained within that block, so neither can be misread as depending on some earlier, unlabeled setup. The E-step block calls FORWARD, forms $\mathcal{L}_{\text{match}}(y_i, \hat{y}_j)$ for every pair from its output, and passes the result to SUBSETSELECTDP to obtain the MAP estimate $\hat{\sigma}$; the per-pair costs belong inside this block, not before it, since they are computed *for* this step and used nowhere else. The M-step block calls FORWARD again,

independently, then holds $\hat{\sigma}$ fixed and evaluates the loss directly from $\hat{p}_{\hat{\sigma}(i)}, \hat{t}_{\hat{\sigma}(i)}$ — never from the E-step’s own cached $\mathcal{L}_{\text{match}}$ values, which play no further role once $\hat{\sigma}$ has been obtained.

Both calls to FORWARD use the identical θ , since nothing updates θ between them within a single invocation of Algorithm 3, so they return identical output; the two blocks calling it independently is a matter of keeping each block’s own dependencies self-contained, not a claim that θ changes in between. An implementation that cares about the resulting redundant computation can call FORWARD once, before either block, and share its output between them, exactly as the algorithm did before the two blocks were separated this way — the two presentations compute the same $\hat{\sigma}$ and the same $\mathcal{L}(\theta)$; only whether FORWARD’s own cost is paid once or twice differs between them.

The stop-gradient step is the discrete analogue of treating a frozen assignment as a constant with respect to θ : $\hat{\sigma}$ enters the loss as data, and only the class probabilities and regressed times of the *matched* slots contribute gradient, exactly as specified in Section 7. Because $\hat{\sigma}(\theta, x) = \arg \min_{\sigma}$ is an argmin over the finite set $\mathfrak{D}_{M,N}$, it is piecewise-constant in θ wherever the minimizer is unique, so this treatment is exact on that region, not an approximation adopted for convenience.

Algorithm 4 Full training procedure

Require: training set $\mathcal{D} = \{(x^{(n)}, y_1^{(n)}, \dots, y_{M_n}^{(n)})\}_{n=1}^{|\mathcal{D}|}$; initial parameters θ_0 ; number of epochs E ; learning rate schedule η ; weights $\lambda_{L1}, \gamma, \{\omega_c\}$

- 1: $\theta \leftarrow \theta_0$
- 2: **for** $e = 1, \dots, E$ **do**
- 3: shuffle $\mathcal{D}$
- 4: **for** each fragment $(x, y_1, \dots, y_M) \in \mathcal{D}$, in shuffled order **do**
- 5: $\theta \leftarrow \text{TRAINSTEP}(x, y_1, \dots, y_M, \theta, \eta, \lambda_{L1}, \gamma, \{\omega_c\})$ ▷ Algorithm 3
- 6: **end for**
- 7: **end for**
- 8: **return** θ

This is ordinary per-fragment stochastic gradient descent: each call to Algorithm 3 both selects $\hat{\sigma}$ for that fragment under the *current* θ and immediately updates θ from the resulting loss, so every fragment sees a matching computed under a slightly different θ than the fragment before it, consistent with $\hat{\sigma}$ being recomputed fresh at every call rather than cached. Minibatching is a natural extension we do not spell out in full: it requires only accumulating $\mathcal{L}(\theta)$ (line 18 of Algorithm 3) across several fragments, each with its own independently-computed $\hat{\sigma}$, before taking a single averaged gradient step, rather than updating θ once per fragment as written here — nothing about $\hat{\sigma}$ ’s own per-fragment computation changes, since it is already fragment-local and detached from the gradient regardless of how many fragments’ losses are summed before the step that consumes it.

9.2 Inference

At inference time there is no ground truth to match against, so the dynamic program of Section 5 plays no role. Each of the N candidates is decoded independently: its predicted class is taken as the $\arg \max$ of $\hat{p}_j(\cdot \mid x; \theta)$, and it is reported as a detected event only if that class is not $\emptyset$ and its probability clears a fixed confidence threshold τ .

Algorithm 5 Inference: decoding beat and downbeat locations

Require: audio window x ; trained parameters θ ; confidence threshold $\tau \in (0, 1)$

```
1:  $B \leftarrow \emptyset$  ▷ output list of detected events
2:  $\{\hat{p}_j(\cdot \mid x; \theta), \hat{t}_j\}_{j=0}^{N-1} \leftarrow \text{FORWARD}(x, \theta)$  ▷ Algorithm 2
3: for  $j = 0, \dots, N-1$  do
4:    $\hat{c}_j \leftarrow \arg \max_{c \in \{\text{DB}, \text{B}, \emptyset\}} \hat{p}_j(c \mid x; \theta)$ 
5:   if  $\hat{c}_j \neq \emptyset$  and  $\hat{p}_j(\hat{c}_j \mid x; \theta) \geq \tau$  then
6:     append  $(\hat{c}_j, \hat{t}_j)$  to  $B$ 
7:   end if
8: end for
9: return  $B$  ▷ already in ascending time order, since  $\hat{t}_0 < \dots < \hat{t}_{N-1}$ 
```

No non-maximum suppression or post-hoc de-duplication step is needed. This follows from two facts that hold regardless of how $z_0, \dots, z_{N-1}$ are obtained from the encoder output: exactly one classification decision is made per candidate index, and $\hat{t}_0 < \hat{t}_1 < \dots < \hat{t}_{N-1}$ holds for every θ and x by equation (1). Together these guarantee that no two reported detections can ever coincide or cross in time. The predicted times are points, not intervals — the relevant property of a point sequence is strict ordering, not disjointness — and the guarantee depends only on this ordering and on there being one classifier per candidate, not on any property of the downsampling operator used to produce z_j .

9.3 Stitching Overlapping Fragments at Inference

Algorithm 5 above decodes one fragment in isolation. For a piece longer than D , the fragments produced by the overlap-and-trim scheme of Section 2 must be decoded separately and combined into one detection list for the whole piece. The obstacle raised in Section 2 was that $\hat{\sigma}$ is a discrete correspondence internal to one fragment’s own candidate indexing, with no meaning across fragments — but that obstacle only applies to $\hat{\sigma}$ itself. It does not apply to the decoded output of Algorithm 5, which is already a list of (class,time) pairs, and a time can always be re-expressed on a shared, absolute axis once a fragment’s start offset is known. Stitching should therefore happen at the level of decoded detections, exactly where Foscarin et al. (2024) stitch theirs, not at the level of $\hat{\sigma}$.

Recall from Section 2 the border duration $\beta \in (0, D/2)$ and the tiling offsets $o_1 = 0, o_2 = D - 2\beta, o_3 = 2(D - 2\beta), \dots$, so consecutive fragments overlap by 2β . Each fragment’s keep region is $[o_k + \beta, o_k + D - \beta]$, except the first fragment’s keep region extends down to $o_1 = 0$ and the last fragment’s keep region extends up to the end of the piece — there is no preceding or following fragment to prefer over the piece’s own boundary.

Algorithm 6 Piece-level inference by stitching overlapping fragments

Require: full piece of duration l ; trained parameters θ ; threshold τ ; fragment duration D ; border β

```
1:  $B_{\text{piece}} \leftarrow \emptyset$ 
2: compute fragment offsets  $o_1, o_2, \dots, o_K$  with stride  $D - 2\beta$  covering  $[0, l]$ 
3: for  $k = 1, \dots, K$  do
4:    $x_k \leftarrow$  the  $D$ -second audio fragment starting at  $o_k$  (padded if it extends past  $l$ )
5:    $B^{(k)} \leftarrow \text{INFERENCE}(x_k, \theta, \tau)$   $\triangleright$  Algorithm 5, run independently per fragment
6:   determine this fragment's keep region  $[\ell_k, u_k]$  (edge exceptions for  $k = 1$  and  $k = K$  as above)
7:   for  $(\hat{c}, \hat{t}) \in B^{(k)}$  do
8:      $\hat{t}_{\text{abs}} \leftarrow o_k + \hat{t} \cdot D$ 
9:     if  $\ell_k \leq \hat{t}_{\text{abs}} \leq u_k$  then
10:      append  $(\hat{c}, \hat{t}_{\text{abs}})$  to  $B_{\text{piece}}$ 
11:     end if
12:   end for
13: end for
14: sort  $B_{\text{piece}}$  by  $\hat{t}_{\text{abs}}$ 
15: return  $B_{\text{piece}}$ 
```

By construction the keep regions of consecutive fragments partition $[0, l]$ with no gap and no overlap, so every absolute time in the piece is the responsibility of exactly one fragment, and a detection surviving the filter is never duplicated or dropped purely because of where the boundary happened to fall.

What this resolves, and what it does not. This closes the gap flagged in Section 2: there is now a fully specified procedure for combining fragment-level output into one piece-level result, built entirely from quantities already defined in this document (Algorithm 5, the fragment offsets of Section 2, and ordinary time arithmetic), with no new machinery over $\hat{\sigma}$ itself required. It does not eliminate the deeper problem underlying it. A true event lying inside fragment k 's keep region is only reported if fragment k 's own $\hat{\sigma}$ happens to detect it; if that fragment's correspondence problem misses or misclassifies it — for reasons unrelated to stitching, such as the reduced context near that fragment's own edge — the event is lost even though a neighboring fragment, for which the same event falls in its discarded border region, might have detected it correctly. This is the same residual risk Foscarin et al.'s frame-level trimming carries, not a new one introduced here; stitching reduces where boundary errors can occur without removing the possibility that one occurs.

The border β is left as a free parameter, exactly as their 120 ms border was: we do not derive its value here. A reasonable starting point, unvalidated, would tie it to the candidate spacing D/N rather than picking it independent of the architecture, since D/N sets the resolution the correspondence problem already operates at; whether this or some other choice performs best is an empirical question outside the scope of this document.

10 Why Downbeat Detection Is Harder, and Formulation Adjustments

10.1 Diagnosis

Beat detection is fundamentally a periodicity-estimation problem: a beat is any point in a train of roughly evenly spaced onsets, and onset-strength features computed directly from the signal correlate strongly with beat locations at essentially any tempo and in essentially any genre. Downbeat detection asks a different question given the beat grid: which beat is beat one of the bar. This is a question about metrical structure — a coarser periodicity of period L (the bar length) nested inside the beat periodicity — and metrical structure is frequently not marked acoustically at all. A snare backbeat on beat two, a bassline anticipating the downbeat, or a passage with no dynamic accent pattern gives a local feature almost nothing to key on for phase within the bar, even when the beat period itself is estimated correctly. Four factors compound this gap in practice. First, the acoustic correlates of downbeat position — harmonic change, bass emphasis, dynamic accent — are weak and inconsistent across genres, unlike the near-universal correlation between onset strength and beat position. Second, downbeats are roughly $1/L$ as frequent as beats, so a downbeat classifier sees far fewer positive examples than a beat classifier even after conditioning on an event being present at all. Third, correctly identifying phase within the bar in principle requires context from several consecutive beats, not just the local neighborhood around one candidate; a feature extractor tuned for local onset salience is, by construction, examining the wrong timescale for this decision. Fourth, in decoders that estimate a meter length explicitly, a meter misestimate does not produce an occasional isolated error — it shifts every downbeat prediction in the piece systematically, since bar phase is a global, not local, quantity.

It is important not to overstate this diagnosis in one respect: the metrical-perception literature indicates that the downbeat is, for human listeners, the most salient position in the metrical hierarchy, and that even musically untrained listeners spontaneously align their motor response to it rather than to a metrically weaker position (Ladinig et al., 2009). Downbeats are not, for a human, an inherently subtle or hard-to-perceive event; the four factors above are difficulties for a discriminative model reading local acoustic features, not evidence that downbeats are weakly marked from a listener’s perspective. This sharpens rather than undercuts the first and third factors: if human listeners identify downbeats reliably despite locally weak or inconsistent acoustic correlates, they are evidently doing so using something beyond local onset salience — top-down integration of harmonic rhythm, phrase structure, and learned genre convention over a timescale of several bars, not an acoustically local marker. A discriminative model with access only to pooled per-candidate features, as in the unmodified architecture of Section 3, has no analogous source of long-range structural information, which is precisely the gap the candidate-level self-attention mechanism of Section 10.2 is intended to narrow.

10.2 A candidate-level self-attention pass for the classification branch

Section 3 assumes a transformer encoder, so each frame feature h_t already has, in principle, access to the whole input window through self-attention before downsampling into z_j . This does not by itself resolve the third factor above: self-attention over raw audio frames is under no obligation to organize the information it aggregates along the beat grid, and the classification head for candidate j reads only the single feature vector z_j , which may not expose bar-periodic structure even when the frame-level features that produced it were globally contextualized. What is missing is not global context per se, but context organized at the right granularity — one token per candidate, aligned

to the beat grid, rather than one token per audio frame.

We address this by adding a second, shallow self-attention block operating over the sequence of pooled candidate features $z_0, \dots, z_{N-1}$, used only by the classification branch:

$$\tilde{z}_j(\theta, x) = \text{SelfAttention}(z_0, \dots, z_{N-1})_j, \quad j = 0, \dots, N-1, \quad (35)$$

with class logits computed from $\tilde{z}_j$ in place of z_j . The regression head is left reading z_j directly, unmodified from Section 3: the argument for keeping the time prediction local is not that it is architecturally confined to a region — it is not, by Section 3 — but that placing $\hat{t}_j$ near a matched ground-truth time is a comparatively local decision to supervise, needing only enough information to disambiguate a single onset’s precise timing, whereas its class, and downbeat status in particular, is a decision that benefits from beat-grid-scale context that z_j alone does not expose. Equation (35) does not disturb the monotonicity guarantee of Section 3, since $\hat{t}_j$ is computed from z_j , not $\tilde{z}_j$, and the ordering $\hat{t}_0 < \dots < \hat{t}_{N-1}$ is unaffected by any transformation applied only to the classification branch. In terms of Algorithm 2, this amounts to inserting $\tilde{z}_j \leftarrow \text{SelfAttention}(z_0, \dots, z_{N-1})_j$ immediately after its own per-candidate feature-extraction loop and computing ℓ_j from $\tilde{z}_j$ in place of z_j , while r_j continues to be computed from z_j unchanged; since both blocks of Algorithm 3 obtain $\hat{p}_j$ only through their own calls to FORWARD, no further change to either block is required.

10.3 Class-specific loss reweighting

The class weight ω_{c_i} introduced in equation (8) directly targets the second factor above. Setting $\omega_{\text{DB}} > \omega_{\text{B}} = 1$ increases the gradient contribution of each downbeat classification error relative to each beat classification error, compensating for the fact that a matched downbeat candidate is encountered roughly $1/L$ as often as a matched beat candidate over a training epoch. This is a direct analogue of the background down-weighting γ already present in the loss, applied one level down, to the imbalance among the active classes rather than between active and inactive candidates.

10.4 A periodicity regularizer for downbeat spacing

The first and fourth factors above call for something the classification loss alone cannot supply: an explicit preference for evenly spaced downbeats, which is the discriminative analogue of the tempo-consistency the generative bar-pointer DBN obtains for free from its transition law. Let $i_1 < \dots < i_K$ index the ground-truth events with $c_{i_k} = \text{DB}$ among $y_1, \dots, y_M$, and let $\hat{\sigma}$ be the matched assignment from Section 7, so that $\hat{t}_{\hat{\sigma}(i_1)}, \dots, \hat{t}_{\hat{\sigma}(i_K)}$ are the predicted times of the matched downbeat candidates. Let

$$\bar{\Delta}(\theta, x) = \frac{1}{M-1} \sum_{i=1}^{M-1} (\hat{t}_{\hat{\sigma}(i+1)}(\theta, x) - \hat{t}_{\hat{\sigma}(i)}(\theta, x)) \quad (36)$$

be the mean spacing among all M matched candidates, beats and downbeats together, serving as a differentiable estimate of the beat period. Define

$$R(\theta, x) = \sum_{k=1}^{K-1} \left(\hat{t}_{\hat{\sigma}(i_{k+1})}(\theta, x) - \hat{t}_{\hat{\sigma}(i_k)}(\theta, x) - L \bar{\Delta}(\theta, x) \right)^2, \quad (37)$$

where L is the meter length (e.g. the time-signature numerator), assumed known at the dataset or track level. This term penalizes deviation of consecutive predicted downbeat spacing from L beat

periods, and is added to the training loss as $\mathcal{L}(\theta) + \lambda_R R(\theta, x)$ with a fixed weight $\lambda_R \geq 0$. Because R depends on the entire matched downbeat sequence rather than a single ground-truth/candidate pair, it cannot be expressed as a per-pair term in $\mathcal{L}_{\text{match}}$ and is instead added after the correspondence $\hat{\sigma}$ has already been fixed, alongside the rest of $\mathcal{L}(\theta)$ in Algorithm 3: concretely, $\lambda_R R(\theta, x)$ is added to the loss line once $\hat{\sigma}$ is available, reusing the $\hat{t}_{\hat{\sigma}(i)}$ values already computed there.

Remark 3. *A further extension, discussed here but not formalized, would add a small global head estimating meter length $\hat{L}$ and bar phase $\hat{\phi}$ from globally pooled features, analogous to a tempo-estimation head, and use $(\hat{L}, \hat{\phi})$ to bias the per-candidate downbeat logits directly rather than only regularizing predicted spacing after the fact. This is a lower-dimensional, more structured decision than N independent three-way classifications, and it is the point at which this discriminative formulation would most closely resemble the phase state of the generative bar-pointer model — a resemblance that should be flagged explicitly, not treated as incidental, if this extension is pursued, per the paradigm-discipline note in Section 11.*

11 Discussion and Scope

This formulation is discriminative throughout: $\hat{t}_j$ and $\hat{p}_j$ are deterministic functions of θ and x produced by a feed-forward network, trained by minimizing (8) directly. It contains no hidden Markov state, no transition density over tempo or phase, and no notion of a bar-pointer position. It should not be presented as an alternative parameterization of, nor combined into the method section of, the generative bar-pointer DBN framework; the two make different independence assumptions and answer different questions about the same signal. Where a comparison is warranted, it belongs in a discussion or related-work section, not in a shared derivation.

The monotonicity device of equation (1) is precisely the cumulative-positive-increment construction used to enforce ordered thresholds in cumulative-link ordinal regression models, applied here to a sequence of instance-specific continuous outputs rather than to a shared set of dataset-level cutpoints; the resemblance is at the level of this one reparameterization device only. Ordinal regression predicts a single discrete label against a shared, dataset-level set of ordered cutpoints, whereas the present model regresses N instance-specific continuous times, each independently classified, with the active subset resolved by selection; the two problems should not be conflated beyond the shared reparameterization trick.

What CTC and DTW actually are. Connectionist Temporal Classification (CTC) is a training method for sequence problems in which the input is much longer than the output and no frame-by-label alignment is given in advance — speech recognition is the standard case, with many audio frames mapping to few output characters. The network emits a distribution over the label alphabet plus a special blank symbol at every input time step; a many-to-one collapsing rule (merging repeated labels, dropping blanks) maps any such raw output sequence down to a final label sequence. Training marginalizes the probability of the correct label sequence over every raw sequence that collapses to it — exponentially many in principle, but computed exactly via a forward-backward dynamic program, because the alignment is constrained to be monotonic: output labels appear in the same order as the input frames that produced them. That monotonicity constraint is precisely what makes the dynamic program tractable, in $O(\text{input length} \times \text{output length})$ time — the same complexity shape as this document’s own construction.

Dynamic Time Warping (DTW) is an older, pre-deep-learning algorithm for comparing two sequences that may differ in speed or timing — two spoken recordings of the same word at different paces, for instance. It finds the minimum-cost monotonic alignment, a warping path, between the

two sequences, where a point in one may stretch to match several points in the other, solved exactly by filling a cost table via dynamic programming — structurally the same device as CTC’s forward algorithm and this document’s own dynamic program of Section 5, though DTW typically allows a few distinct step types (diagonal, horizontal, vertical) rather than only skip and match.

The subset-selection reduction of Section 4 is structurally related to the monotonic alignment assumption underlying Connectionist Temporal Classification and to Dynamic Time Warping: both replace a search over a large combinatorial space of correspondences with a search constrained by a fixed order, exploitable by the same $O(NM)$ -type dynamic program, for the same complexity-reduction reason. The connection is closer than shared complexity alone: Section 5 establishes that this construction is, precisely, a scoring lattice — a deterministic, fixed set of legal moves through the (i, j) grid, with all learned weight carried by local emission scores and none by transition weights — which is exactly the structure CTC’s and DTW’s own dynamic programs share, not merely an analogous complexity class.

This is also, more directly, an instance of the broader pattern of latent alignment via dynamic programming that CTC, DTW, and, in a related but non-identical form, latent subword segmentation for machine translation all belong to. CTC’s own blank symbol is the closer analogy for an unmatched candidate here: a position in the alignment that emits nothing and contributes nothing to the final decoded sequence, exactly the role an unmatched candidate plays in σ . He, Haffari, and Norouzi (2020) treat subword segmentation as a related but structurally different case worth distinguishing precisely, rather than folding in without comment: their segmentation is dense, in that every character belongs to some token and the characters interior to a token jointly determine what that token is, whereas the alignment here is sparse, since an unmatched candidate contributes nothing to any neighboring event’s identity and no span between two matched candidates carries meaning of its own. This document’s construction is therefore best read as the sparse, CTC-blank-like member of the broader latent-alignment-via-dynamic-programming family — sharing CTC’s and DTW’s emission-only structure precisely, established above, and sharing with subword segmentation only the general marginalize-via-DP device, not its dense semantics.

12 Comparison to a Dense Frame-Classification Baseline

Foscarin, Schlüter, and Widmer’s Beat This! (2024) is a trained, empirically validated discriminative beat and downbeat tracker that, like the present formulation, dispenses with DBN postprocessing; it is instructive to compare the two, not because they solve the same problem the same way, but because the differences trace directly to a single architectural choice. We emphasize at the outset that Beat This! reports state-of-the-art F1 scores across eighteen datasets with full ablations, whereas the present document proposes and analyzes a formulation without a trained model or empirical results; the comparison below is architectural, not a claim of comparative performance.

Beat This! processes 30-second windows through a convolution-and-transformer frontend into a transformer with full self-attention over time, producing two independent per-frame binary logit sequences — beat and downbeat — at the frame rate of the input spectrogram (50 frames per second). Beat locations are recovered by picking local-maximum frames above a probability threshold. Annotation timing imprecision is handled by max-pooling predictions over a small window before computing the loss against frame-synchronous binary labels, so that only the largest nearby prediction is penalized and small shifts go unpunished; class imbalance is handled by weighting the positive-frame term of the binary cross-entropy by the ratio of negative to positive frame counts. Because beat and downbeat are independent output channels, thresholding can produce a downbeat with no coincident beat prediction; a “Sum Head” that adds the downbeat logit into the beat logit

before thresholding reduces, but does not eliminate, this inconsistency, and a final post-processing step snaps every downbeat to its nearest beat prediction to guarantee a musically valid output.

The central architectural difference is that Beat This! classifies every frame, while the present formulation regresses a fixed, tempo-bounded number of candidates. This single difference explains most of what follows. Dense per-frame classification needs no correspondence problem: the frame index is the time, so there is nothing analogous to Section 4’s subset selection or Section 5’s dynamic program to solve, and the identity of which output corresponds to which ground-truth event is never ambiguous. The present formulation’s correspondence machinery exists specifically because N candidates without frame-level identity must be matched to M ground-truth events, a problem dense classification sidesteps by construction rather than by solving. A second consequence is where output consistency comes from: Beat This!’s beat and downbeat channels can disagree because they are independently thresholded outputs, requiring the Sum Head and the nearest-beat correction described above as a remedy applied after the fact; in the present formulation a candidate carries a single class label from $\{\text{DB}, \text{B}, \emptyset\}$, so a candidate cannot simultaneously be and not be a beat — consistency is a consequence of the output representation, not a learned or post-processed correction. A third consequence concerns time resolution: Beat This!’s resolution is fixed by the spectrogram hop size regardless of tempo, while the present formulation’s candidate count is set from a tempo bound (Section 2) independent of frame rate, trading a large fixed-resolution output for a smaller, tempo-scaled one with an explicit non-crossing guarantee (equation (1)) that a per-frame classifier has no analogue of.

The two systems address annotation timing imprecision by different means aimed at the same problem: Beat This!’s max-pooled loss is a procedural device with a window size chosen empirically, while the derivation in Section 4 states an explicit observation model and arrives at an interpretable, in-principle-learnable scale parameter (Section 4.1) rather than a fixed pooling width; neither approach has been shown superior to the other, and the comparison should be read as a difference in what each choice makes explicit, not as an argument for one over the other absent empirical evidence. Finally, Beat This! explicitly identifies the same weakness diagnosed independently in Section 10.1: they report that removing DBN postprocessing degrades continuity metrics specifically because their loss treats each beat individually and does not penalize non-periodic predictions, and list developing a loss that enforces periodicity during training as future work. This is a point of genuine convergence rather than contrast — an empirical finding on a trained, evaluated system arriving at the same diagnosis reached mechanistically in Section 10.1, and the periodicity regularizer of equation (37) is a candidate answer to precisely the gap they name, though it remains unvalidated here.

References

- [1] S. Böck, F. Krebs, G. Widmer. *Joint Beat and Downbeat Tracking with Recurrent Neural Networks*. ISMIR 2016.
- [2] A. Graves, S. Fernández, F. Gomez, J. Schmidhuber. *Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks*. ICML 2006.
- [3] H. Sakoe, S. Chiba. *Dynamic Programming Algorithm Optimization for Spoken Word Recognition*. IEEE Transactions on Acoustics, Speech, and Signal Processing, 1978.
- [4] O. Ladinig, H. Honing, G. Háden, I. Winkler. *Probing Attentive and Preattentive Emergent Meter in Adult Listeners without Extensive Music Training*. Music Perception, 26(4), 2009.

- [5] F. Foscarin, J. Schlüter, G. Widmer. *Beat This! Accurate Beat Tracking Without DBN Post-processing*. ISMIR 2024.
- [6] K. Nigam, A. McCallum, S. Thrun, T. Mitchell. *Text Classification from Labeled and Unlabeled Documents Using EM*. Machine Learning, 39, 2000.
- [7] J. Lafferty, A. McCallum, F. Pereira. *Conditional Random Fields: Probabilistic Models for Segmenting and Labeling Sequence Data*. ICML 2001.
- [8] G. Celeux, G. Govaert. *A Classification EM Algorithm for Clustering and Two Stochastic Versions*. Computational Statistics & Data Analysis, 14(3), 315–332, 1992.
- [9] X. He, G. Haffari, M. Norouzi. *Dynamic Programming Encoding for Subword Segmentation in Neural Machine Translation*. ACL 2020.